

JYTRA TECHNOLOGY SOLUTIONS · CONFIDENTIAL · 2025–2028

ActCAD Re-Architecture

Strategic & Engineering Documentation

3-year programme to replace the IntelliCAD engine
with a first-party engine on ODA SDKs and the ACIS 3D kernel.

Jytra Technology Solutions Pvt. Ltd.
East Godavari, India

Contents

MASTER RE-ARCHITECTURE PLAN

ActCAD Re-Architecture Plan

1. Goal	
2. Key architectural decisions	
3. Module decomposition (C++ core)	
4. Stack at a glance	
5. Phasing — native-Windows-first, web in parallel	
Phase 1 — Foundation (months 0–12)	
Phase 2 — Production v1 (months 12–24)	
Phase 3 — Parity-or-better (months 24–36)	
6. What we deliberately don't do	
7. Risks and mitigations	
8. Migration story for existing customers	
9. Feasibility spike — 4–6 weeks BEFORE committing to the 3-year plan	
10. Open decisions for legal / commercial review	
11. Industry adoption — who else picked these and why	
11.1 ODA SDKs (Drawings, Visualize, IFC, BIM, MCAD, inWEB) — direct membership	
11.2 ACIS (Spatial / Dassault) — 3D kernel	
11.3 ODA MCAD SDK — translator, not kernel	
11.4 Qt 6 — desktop shell	
11.5 C++ vs Rust for the engine core	
11.6 WebGPU + WebAssembly — browser CAD	
11.7 ODA inWEB (Drawings inWEB, Visualize inWEB)	
11.8 CMake + Emscripten — C++ → WASM toolchain	
11.9 Summary verdict matrix	
12. Collaboration architecture in detail	
12.1 What the leaders actually shipped	
12.2 Why generic CRDTs lose on DWG	
12.3 ActCAD's two-tier model — detailed	
12.4 What we copy, what we avoid	
12.5 Phasing for collaboration	
13. Extensions and marketplace strategy	
13.1 What the leaders actually shipped	
13.2 Architecture patterns	

13.3 ActCAD's extension model	
13.4 Marketplace policy	
13.5 What to avoid (concrete anti-patterns)	
14. AI and agent strategy in detail	
14.1 What the leaders actually shipped	
14.2 The two architectural patterns	
14.3 Where the current generation goes wrong	
14.4 ActCAD's three-step strategy	
14.5 Agent tool design rules	
14.6 What we avoid	
15. Cost of usage — third-party stack, shells, tools	
15.1 Unified pricing reference	
15.1.1 ACIS royalty model — per-deployment vs per-seat	
15.2 Modeled annual budget envelopes	
15.3 What's modeled vs what's opaque	
15.4 Commercial commitments and cost-driven decisions	
16. Memory architecture — designing for headroom from day 1	
16.1 The five layers of memory in a live CAD session	
16.2 Eight architectural commitments that keep responsiveness flat as drawings grow	
16.3 Where bottlenecks actually appear in shipping CAD products, and the architectural answer to each	
16.4 What to measure, and how to keep it from regressing	
16.5 Tools and people to bring in	
16.6 What we deliberately don't do	
16.7 Bottom line in one paragraph	
17. Next steps	

ARCHITECTURE OVERVIEW & IMPLICATIONS

ActCAD — Architecture, Pros/Cons, and Re-Architecture Implications

Context	
1. What ActCAD Actually Is	
1.1 Layered Stack (bottom → top)	
1.2 How a Drawing Flows Through the Stack	
2. Pros of the Current Architecture	
3. Cons / Architectural Debt	
3.1 Strategic (the expensive ones)	
3.2 Technical	
4. Implications for a Re-Architecture	
4.0 Why the engine decision is forced — three lived pains	

4.1 The four big decisions	
4.2 Risks to manage	
4.3 What to ask of the internal material (when it arrives)	
5. Verification	
Sources	

INDUSTRY OUTLOOK

CAD Industry Outlook — ActCAD's Segment, 2–5 Year Horizon

Scope	
1. Five forces reshaping the segment	
1.1 AI assistance becomes table stakes (12–24 months)	
1.2 Cloud + web + mobile parity becomes baseline (24–36 months)	
1.3 BIM / CAD convergence moves downmarket (24–60 months)	
1.4 Pricing models fragment, perpetual partially returns (ongoing)	
1.5 Digital twin + IoT enters the SMB MEP conversation (36–60 months)	
2. Where ActCAD's user base is going	
3. Geographic forecast	
4. What this means for the re-architecture	
Sources	

MEMORY ARCHITECTURE — ENGINEERING REFERENCE

ActCAD Memory Architecture

Table of contents	
1. Why this document exists	
2. Goals and non-goals	
2.1 Goals	
2.2 Non-goals	
2.3 Success criteria	
3. The CAD memory model in plain terms	
3.1 Open-once, edit-in-memory, save-back	
3.2 Why naive in-memory blows up	
3.3 What “the drawing in memory” actually contains	
4. The five-layer model	
4.1 Layer 1 — DWG database (db)	
4.2 Layer 2 — ACIS kernel bodies (geom via KAL)	
4.3 Layer 3 — Tessellation cache (render)	
4.4 Layer 4 — Spatial indexes (db + render)	
4.5 Layer 5 — Undo / op-log (db)	

4.6 Ownership matrix	
4.7 Eviction order under pressure	
5. The eight commitments — full detail	
5.1 Single mutator: db owns all drawing state	
5.2 MVCC: reads never block writes (and writes never block reads)	
5.3 Spatial index built at load time	
5.4 Lazy materialization of ACIS bodies	
5.5 GPU-resident, evictable tessellation	
5.6 Delta-based undo	
5.7 Declared memory budget; fail loudly, never silently swap	
5.8 8 ms UI-thread budget	
6. Concurrency model in depth	
6.1 Threads we use	
6.2 The transaction queue	
6.3 Snapshot mechanics (copy-on-write)	
6.4 The op-stream as the universal substrate	
7. The memory-budget machinery	
7.1 Budget declaration and discovery	
7.2 Sub-budgets per layer	
7.3 Eviction protocol	
7.4 Pressure signals and OS APIs	
7.5 Loud-failure UX	
8. The Kernel Abstraction Layer (KAL)	
8.1 What the KAL hides	
8.2 Opaque handle convention	
8.3 Lazy materialization contract	
8.4 Memory ownership rules across the KAL boundary	
9. Integration with other modules	
9.1 render and the tessellation cache	
9.2 cmd and transaction boundaries	
9.3 agent and read-only snapshots	
9.4 net and op-stream replication	
9.5 script and plugin — sandboxed access patterns	
10. WASM and the browser shell	
10.1 The 4 GB browser memory ceiling	
10.2 Allocator implications	
10.3 Tessellation differences (WebGPU vs native)	
10.4 ACIS in the browser — translator path only	
11. Mobile implications (Phase 3)	
11.1 iOS app memory limits	

11.2 Android Vulkan considerations	
11.3 When to refuse to open	
12. Failure modes and recovery	
13. Telemetry and observability	
13.1 What we log	
13.2 What we never log (PII)	
13.3 The “performance health” customer-visible badge	
14. Testing strategy	
14.1 The DWG corpus	
14.2 Synthetic loops	
14.3 Long-session soak	
14.4 Sanitizers in CI	
15. CI gates — exact specifications	
15.1 Open-time gate	
15.2 Frame-time gate	
15.3 Memory ceiling gate	
15.4 Allocation count gate	
15.5 Leak gate	
15.6 Soak gate	
15.7 Failure handling — what counts as a flake	
16. Tools	
16.1 Tracy (always-on)	
16.2 Intel VTune (deep dives)	
16.3 Heaptrack and Massif	
16.4 Visual Studio Diagnostics + WPA	
16.5 AddressSanitizer + LeakSanitizer	
16.6 mimalloc / jemalloc as system malloc	
17. Expertise sourcing	
17.1 Spatial CTC	
17.2 External performance engineers	
17.3 Peer back-channels	
18. Anti-patterns — what we will not allow	
18.1 Snapshot-based undo	
18.2 Eager attribute loading	
18.3 Linear-scan hit-testing	
18.4 Synchronous file I/O on the UI thread	
18.5 Allocator-of-the-week thrash	
18.6 Plugin direct-db access	
18.7 GC-based memory management in the engine hot path	
18.8 Hidden caches	

19. Migration from IntelliCAD patterns

19.1 What we inherit (nothing)

19.2 Patterns we deliberately don't copy

19.3 What to teach engineers coming from IntelliCAD

20. Phase-by-phase rollout

20.1 Phase 1 (months 0–12) — foundations

20.2 Phase 2 (months 12–24) — co-edit + AI integration

20.3 Phase 3 (months 24–36) — mobile + scale-out

21. Open questions

22. Glossary

23. References

ODA SDKs

ACIS

MVCC and database concurrency

Tooling

Companion documents

Master Re-Architecture Plan

ActCAD Re-Architecture Plan

Status: Proposal, gated on the feasibility spike in §9. Read alongside `docs/architecture-overview.md` (current stack and why it has to be replaced) and `docs/industry-outlook.md` (where the segment is going).

1. Goal

Replace the IntelliCAD engine with a first-party engine built directly on **ODA SDKs and the ACIS 3D kernel (Spatial / Dassault)**, over a 3-year horizon. By end of year 3, ActCAD on the new stack must equal or beat today's IntelliCAD-based ActCAD on every dimension that matters to customers (performance, platform reach, AI assistance, BIM, extensibility), while existing ActCAD continues to ship through Phase 2 to protect revenue.

The architecture must be **future-proof on three axes that the current stack is not**: cross-device (desktop + web + mobile), AI-native (first-party MCP/agent surface inside the engine), and ours to refactor (no consortium gating).

2. Key architectural decisions

#	Decision	Choice	Rationale
2.1	Foundation libraries	ODA direct (Drawings, Visualize, IFC, BIM SDKs, MCAD, Civil, Scan-to-BIM, inWEB family). Drop ITC / IntelliCAD entirely.	One vendor relationship covering DWG + BIM + IFC + renderer + web. Royalty-free under ODA membership. Eliminates the consortium cadence problem.
2.2	3D kernel	ACIS (Spatial / Dassault) primary , full commercial OEM contract. All kernel access goes through the KAL so the engine never depends on ACIS-specific types in public headers. ODA MCAD is not the kernel — it's a translator for reading and writing SolidWorks / Inventor / CATIA files (see §11). Gated on §9 spike item 1 (ACIS round-trip fidelity vs AutoCAD on customer DWGs) and spike item 1b (ACIS commercial scoping under NDA).	AutoCAD's 3D solids are stored as ASM (Autodesk Shape Manager), which is a fork of ACIS that Autodesk took private around 2001. A DWG with 3D solids contains ACIS/ASM SAT (Standard ACIS Text) blobs as <code>AcDb3dSolid</code> entities. For lossless round-trip of AutoCAD-origin 3D solids — the central value prop for AutoCAD migrators — ACIS is the natural kernel: same lineage, same topology model, same tolerance semantics. Translating ACIS → any-other-kernel → ACIS introduces drift on edges, fillets, and booleans that customers will see and reject. ActCAD already uses ACIS today through the IntelliCAD stack; this decision preserves geometric continuity for the existing customer base while putting the contract directly in our name.
2.3	Shell strategy	Shared C++ core compiled to native + WASM. Qt 6 native shells (Win / macOS / Linux) on the commercial license (not LGPL — see §15.4). TypeScript + React + WebGPU browser shell. Mobile deferred to Phase 3.	Web-only Tauri / Electron is rejected for the drawing window — CAD pros need native menus, drag-drop with the OS, accessibility APIs, and printer integration. Qt is what Autodesk Maya and most pro DCC apps use. Tauri stays as the right tool for a thin launcher / license app, not the editor. Qt commercial is required for static linking, modifications kept private, and iOS code-signing — all of which apply to a closed-source commercial product.
2.4	Engine language	C++20 primary (interop directly with ODA's C++ SDKs — no FFI tax on inner-loop database calls). Rust selectively for <code>net</code> , <code>script</code> (host), <code>agent</code> (MCP), and new geometry algorithms where memory safety compounds. No Rust in db or render hot paths.	A CAD inner loop touches millions of entities per regen; the FFI tax of a Rust-primary core against a C++ SDK would be unacceptable. Rust earns its place where the boundary is narrow and the safety win compounds.
2.5	Extension APIs	AutoLISP-compatible runtime (migration story for the customer ecosystem). Modern: Python (embedded), TypeScript (in-app + browser), .NET 8+ (cross-platform via NativeAOT). Dropped: SDS, ADS, DIESEL, VBA, COM. .NET surface is	LISP is the AutoCAD-ecosystem migration moat. The rest are legacy maintenance contracts with shrinking usage. The new .NET surface is opinionated and modern; pretending it's ObjectARX is a trap.

#	Decision	Choice	Rationale
		new — not ObjectARX-compatible (set this expectation early).	
2.6	AI / agent surface	MCP server inside the engine, wrapping the cmd module only (never the db directly). Two MCP endpoints mirroring Autodesk Fusion 2026: a local MCP (requires ActCAD running, executes commands inside the host transaction) and a remote MCP data server (cloud, no host required, read-only DWG metadata + sheet / layer queries). Public commitment: every MCP tool call resolves to exactly one transaction in the host undo stack. No competitor (Autodesk Revit 2027, Fusion 2026, BricsCAD, ARES A3) has publicly documented this — it's a defensible differentiator.	The hard part of an agent surface for CAD is the <i>transactional command boundary</i> , not exposing the database. Revit 2027 MCP is Tech Preview without documented undo semantics; reviewers have already complained about half-finished features. Owning the transaction story publicly is both the right engineering and the right marketing.
2.7	Cloud / co-edit	Two-tier server-authoritative model. Tier 1 (geometry): serialized op-log over the engine's mutation stream, server-validated for referential integrity (door-to-wall, dim-to-entity), optimistic UI. Tier 2 (annotations, comments, markups, layer properties, sheet titles): Figma-style last-writer-wins per property with fractional indexing for ordering. Presence + selection locks at the UI layer. Not a generic CRDT.	The industry has converged here. Onshape uses serialized feature-level operations with branch-and-merge (USPTO 10,691,844). Figma explicitly rejected both OT and pure CRDT for LWW-per-property because each field updates independently and "merge text by character" isn't a real need. Yjs / Automerge OOM at the millions-of-entity scale of a real MEP DWG; recent academic work (geometry-aware CRDTs, MDPI 2025) confirms generic CRDTs lose intent on spatial documents. ARES Kudo ships closer to soft-lock + sync today — we can do better by going server-authoritative op-log on geometry from day 1.
2.8	License seam	The boundary between local-perpetual code and cloud-subscription code is a build-time module boundary , not a runtime feature flag.	Perpetual remains the moat; cloud / AI is an additive subscription. The architecture has to make this monetization model physically clean rather than glued on later.
2.9	Build system	CMake for C++ (industry standard, plays well with ODA's build pipeline). Emscripten for the WASM target. Cargo for the Rust crates, linked through a stable C ABI.	No exotic build tooling. The team will be onboarding from a C++ / IntelliCAD background.

3. Module decomposition (C++ core)

Ten modules with strict contracts. The only module that mutates state is `db` ; the only module the agent talks to is `cmd` .

1. `db` — drawing database. Canonical entity model, transactional mutation API, undo log, handle table, layer / block tables. Emits a typed op-stream on every commit. **Only mutator.**
2. `geom` — geometry primitives and the Kernel Abstraction Layer (KAL) over ACIS. Pure functions, no state. **No ACIS ENTITY type appears in any header outside this module** — opaque handles only. The KAL is what keeps a kernel swap technically possible (6–12 weeks of focused work) if business conditions ever require it; it is not a hedge against ACIS being the right call today.
3. `render` — view + render-abstraction over ODA Visualize and a future native backend. Read-only consumer of `db` ; subscribes to the op-stream for invalidation.
4. `cmd` — command bus. Every user action, LISP call, .NET call, MCP call flows through here. Owns parsing, prompts, validation, transaction begin / commit, undo grouping. **The single seam the AI agent talks to.**
5. `script` — scripting host. Embeds CPython (native) / Pyodide (WASM), the LISP interpreter, and the JS bridge. All three reach `cmd` , never `db` directly.
6. `plugin` — plugin host. ABI versioning, capability negotiation, sandbox policy (stricter for WASM). API parity across native / WASM, **not** binary parity.
7. `net` — sync, document service client, auth, telemetry. Rust crate, exposed via stable C ABI.
8. `agent` — MCP server, tool catalog, permission scoping, dry-run / rate limiting. Wraps `cmd` . Rust crate.
9. `platform` — file I/O, fonts, clipboard, printer, GPU surface acquisition. **The only module with `#ifdef PLATFORM` .**
10. `ui-bridge` — thin C ABI / FlatBuffers boundary the Qt shell and the React / WASM shell both consume. **No UI code here.**

Communication: synchronous in-process calls within native; FlatBuffers over `postMessage` for WASM ↔ JS; the `db` op-stream is the universal invalidation / sync substrate (also fed to the co-edit network layer and the agent for context updates).

4. Stack at a glance

Layer	Choice
DWG / DXF + database	ODA Drawings SDK
Rendering	ODA Visualize. Native backends: DirectX 11 (Windows default) , DirectX 12 , Vulkan (cross-platform) , Metal (macOS / iOS) . Web: Visualize inWEB → WebGPU (WebGL2 fallback). All driven through our <code>render</code> module's abstraction layer so backends are swappable.
3D Kernel	ACIS (Spatial / Dassault) primary, KAL-abstracted. Commercial OEM contract — initial fee + per-seat / per-deployment royalty + annual maintenance + component-module fees (see §15).
MCAD format I/O	ODA MCAD SDK (SolidWorks / Inventor / CATIA / NX / Creo translators) — read-only initially
BIM / IFC	ODA IFC SDK + BimRv / BimNw extensions
Engine language	C++20, selective Rust
Build	CMake + Emscripten + Cargo
Desktop shell	Qt 6 (commercial-vs-LGPL TBD per §10)
Browser shell	TypeScript + React + WebGPU (WebGL2 fallback)
Mobile shell	Native (Swift / Kotlin) over the C++ core via FFI — Phase 3
Scripting	Python (CPython / Pyodide), TypeScript, .NET 8+ NativeAOT, AutoLISP shim
Agent surface	MCP server in <code>agent</code> , wrapping <code>cmd</code>
Cloud co-edit	Two-tier: server-authoritative op-log (geometry) + LWW per property (annotations / metadata)
Document service / auth / billing	TBD per §10 — leaning Clerk + Stripe + self-hosted document service

5. Phasing — native-Windows-first, web in parallel

Important inversion from the obvious plan: the obvious move is web-first because the industry is going there. For ActCAD's installed base specifically — Windows-desktop SMBs in India / SEA who pay perpetual and run on hardware where browser WebGPU is unreliable — a web-only beta would produce no useful signal. The Phase-1 daily-driver target is native Windows; web ships in parallel as viewer / markup first, then full editor.

Phase 1 — Foundation (months 0–12)

- Engine skeleton on ODA Drawings + Visualize, with the module decomposition in §3.
- DWG open / edit / save with full ODA fidelity. Round-trip corpus exists and gates every PR.
- 2D drafting commands: line, polyline, circle, arc, hatch, dim, layer, block, OSnap.

- Native Windows Qt shell shipped to a willing-customer cohort by month 12.
- Parallel **viewer + markup** browser shell on the same engine, shipped at month 12.
- LISP runtime spike against a measured customer-script corpus; coverage target = top 80th percentile of customer scripts running.
- MCP server in `agent` exposing ~30 commands; AI assistant skeleton with dry-run + undo + permission scoping.
- IntelliCAD-based ActCAD continues to ship in parallel.

Phase 2 — Production v1 (months 12–24)

- macOS and Linux Qt shells at parity with Windows.
- Browser shell promoted from viewer to full editor.
- LISP shim coverage extended to 95th percentile of measured corpus; migration tool ships for the long tail.
- Light 3D (basic solids, view, navigation) via ACIS through the KAL.
- Cloud document service with real-time co-edit on 2D drawings (op-log path, not CRDT).
- Plugin host stabilizes; Architecture vertical reshipped as the first plugin.
- AI assistant moves to GA: prompt-driven drafting, AI-generated blocks, command recommendations.
- New product GA at end of month 24. IntelliCAD ActCAD continues to ship in parallel.

Phase 3 — Parity-or-better (months 24–36)

- Full 3D + BIM-lite (IFC import / export, parametric architectural objects).
- MEP and Electrical verticals reshipped as plugins; digital-twin / IoT extension hooks.
- Mobile (iOS / Android) shells over the same C++ core.
- End-to-end rendering pipeline tuning — now ours to own.
- IntelliCAD ActCAD enters sunset on a published timeline.

6. What we deliberately don't do

- Don't ship web-first to the existing installed base.
- Don't promise *binary* plugin parity across native and WASM — only API parity.
- Don't use a generic CRDT for DWG co-edit.
- Don't bundle SDS / ADS / DIESEL / VBA / COM.
- Don't try to be ObjectARX-compatible on the .NET surface.
- Don't let Rust into `db` or `render` hot paths.
- Don't sign the ACIS contract before §9 spike item 1 confirms round-trip fidelity on customer DWGs and spike item 1b returns a written term sheet from Spatial.
- Don't treat ODA MCAD as a B-rep modeling kernel — it's a translator for MCAD file formats only.
- Don't ship chat-in-canvas as the primary AI UX (Hypar pivoted away from text-to-BIM for this reason — see §14).
- Don't chase Bernini-style generative-CAD demos until editable, parametric round-tripping is proven.

- Don't use Yjs / Automerge for the geometry plane — confine generic CRDTs to non-spatial metadata only.
- **Don't let ACIS types leak past the KAL.** Every public header outside `geom` uses opaque handles. The KAL is what makes the engine survive kernel-vendor risk; bypassing it for “just this one feature” erodes the only insurance we have.
- **Don't ship to customers without royalty-reporting infrastructure in place.** The ACIS DELA requires accurate seat / deployment counts; building this after GA is a contractual exposure we don't take.

7. Risks and mitigations

Risk	Mitigation
ACIS round-trip fidelity vs AutoCAD — customer DWGs must open in new ActCAD, edit, save, and re-open in AutoCAD with no visible drift	§9 spike item 1 tests this on 50 real customer DWGs before signing the contract; KAL preserves a kernel-swap escape hatch if a regression is discovered post-GA
ACIS commercial exposure — opaque pricing, royalty per seat, change-of-control risk (Dassault owns SolidWorks, a competitor)	§9 spike item 1b returns a written term sheet under NDA; commercial / legal review of the DELA + change-of-control + business-continuation clauses; source escrow option negotiated; module list scoped before signing so component fees don't surprise
ACIS WASM viability — no public production WASM build of ACIS; if browser shell needs kernel-grade editing, this is a custom engineering conversation	§9 spike item 2 evaluates the inWEB + read-only-ACIS-translator path for the browser shell; full ACIS in WASM stays a Phase-2/3 question if customer demand requires it
inWEB ≠ same binary as native — likely two builds with shim parity, not one bundle	§9 spike item 2 measures the actual delta; messaging adjusted accordingly
LISP compatibility is multi-person-year , not a 12-month spike	Define “compatibility” as a measured percentile of a real customer-script corpus before kickoff; ship migration tool for the tail
MCP without a transactional boundary corrupts drawings	<code>agent</code> wraps <code>cmd</code> only; never <code>db</code> . Permission scoping, dry-run, rate limit from commit 1
Cadence trap re-emerges with ODA	Renderer escape valve: <code>render</code> is an abstraction over Visualize, so a native Vulkan backend can be slotted in if Visualize stalls
Qt licensing surprises	Commercial license locked (§15.4); §9 spike item 6 negotiates the multi-year quote, not the licensing model
Two parallel product lines for 24 months is an org risk, not a tech risk	Support / engineering / marketing org plan written separately; clear customer messaging from day 1
ui-bridge ABI churn breaks plugins and shells	Version the ABI from commit 1; never break, only extend
DWG handle stability across save / load (customer xdata uses handles)	Lock handle semantics before the save format is frozen in Phase 1
CRDT-vs-op-log is decided on marketing, not on a DWG prototype	§9 spike item 7 prototypes 2-user co-edit on a real drawing before locking the design

8. Migration story for existing customers

- IntelliCAD-based ActCAD continues to ship through Phase 2; sunset date is announced when new product reaches GA.
- LISP migration tool ships in Phase 2 with coverage of the measured customer-script corpus.

- Strategic customer .NET / IRX scripts are migrated by a **paid migration service** delivered by the same team that built the new APIs.
- File formats: full DWG round-trip parity from Phase 1, validated against a corpus that includes real customer drawings.
- Pricing: existing perpetual licenses get an upgrade path to the new product's perpetual tier; AI / cloud features are the new subscription line.

9. Feasibility spike — 4–6 weeks BEFORE committing to the 3-year plan

Seven validations, in priority order. **If items 1, 2, 3, or 5 fail, the plan changes before kickoff, not at the Phase-2 boundary.**

#	What	Pass criterion
1	ACIS round-trip fidelity test vs AutoCAD. Take 50 customer DWGs containing ACIS / ASM B-rep solids. Open through ACIS, edit (move / boolean / fillet), save, re-open in AutoCAD. Measure geometric drift on volume / surface area / bounding box, boolean robustness, fillet survival, attribute preservation.	No visible drift, <0.001 unit tolerance on volume / area, identical entity handle set, booleans pass on ≥98% of customer solids. If this passes, ACIS is locked for the 3-year plan and the contract is signed. If it fails, the assumption that direct-ACIS gives lossless AutoCAD fidelity is wrong and we re-open the kernel decision.
1b	ACIS commercial scoping under NDA. Procurement + legal track running parallel to spike item 1. Get written term sheet from Spatial naming: module list (Local Ops, Healing, Defeaturing, Polyhedral, etc.); royalty model — per-deployment is the ask, per-seat is the documented fallback only (see §15.1.1 for the comparison math, ~10× cost variance, and full negotiating sequence); WASM / Linux / macOS SKU availability; source-escrow option; DELA full text; change-of-control + business-continuation clauses; support-hour bundle.	Signed term sheet in hand before Phase-1 kickoff. Module list scoped to ActCAD's actual use case (no surprise component fees later). Per-deployment royalty locked as the primary ask per §15.1.1; per-seat accepted only if the 5-year amortized cost beats the per-deployment alternative Spatial would have offered. Volume tier breakpoints + annual minimum floor + carve-outs (trials / education / internal QA) all in writing.
2	Render the same 50 DWGs through native Visualize (Vulkan) <i>and</i> through Visualize inWEB (WebGPU); compare frame time on a 250k-entity drawing	inWEB ≤ 3× slower than native
3	Spike the <code>ui-bridge</code> C ABI / FlatBuffers seam: drive 10 commands from both a Qt shell and a React + WASM shell against a single <code>cmd</code> implementation	Seam is one source of truth; no shell-specific branching in <code>cmd</code>
4	Run a real customer LISP script (largest one a top-5 customer uses) through a minimal LISP interpreter on top of <code>cmd</code>	Script runs and produces visually identical output to current ActCAD
5	Stand up an MCP server exposing <code>cmd</code> to Claude / GPT; have it execute “draw a 3-bed apartment plan from this brief”	Command surface is agent-shaped, not human-shaped; no drawing corruption
6	Negotiate Qt 6 commercial multi-year quote at Jytra's actual revenue tier (Small Business almost certainly not eligible). Target 15–35% below first quote per Vendr-reported norm.	Signed quote in hand for App Dev Enterprise + mobile / WASM add-ons covering Phase-1 + Phase-2 headcount, with a price-protection clause for Phase-3 growth.
7	Reproduce 2-user co-edit on a 2D drawing with a stub op-log over WebSocket (not CRDT)	Op-stream design supports replication; conflict semantics on a real DWG operation are sane

10. Open decisions for legal / commercial review

ActCAD is a closed-source commercial product. Where a license has a free open-source path *and* a commercial / paid path, commit to the path that is unambiguously safe for closed-source distribution and standard commercial practice (static linking, code-signed binaries, modifications kept private). The locked / open status of each item below reflects that posture.

- **Qt 6 — LOCKED to commercial.** Static linking, code-signed iOS / macOS distribution, and keeping our Qt modifications private are all required; LGPL v3 does not support those. The remaining open work is *quote negotiation*, not the licensing model — see §9 spike item 6 and §15.4.
- **ACIS — LOCKED as the 3D kernel choice, commercial terms OPEN.** Spatial / Dassault commercial OEM contract. Decision rationale is locked (AutoCAD ASM $\supset$ ACIS lineage $\rightarrow$ lossless DWG fidelity); commercial *terms* (initial fee, royalty model, module list, DELA specifics, change-of-control protection, source escrow) are open pending §9 spike item 1b. **Do not sign before the spike returns a term sheet.**
- **ODA — LOCKED to Sustaining from day 1**, Founding considered in Phase 3 for source / Git access. Limited Commercial's 100-seat cap is a footgun at our scale; Sustaining provides Web/SaaS (inWEB) redistribution rights without per-seat royalty.
- **Visual Studio — LOCKED to Enterprise** for C++ inner-loop devs (advanced profiling, IntelliTrace). Professional for the rest.
- **Document service / auth / billing stack** — recommended Clerk + Stripe + self-hosted document service. **Open** pending data-residency review for EU / India customers.
- **Customer-facing .NET API scope** — explicitly *not* ObjectARX-compatible. **Open** pending announcement / migration messaging review.
- **IntelliCAD-ActCAD sunset date** — **open**, driven by Phase 2 GA quality, not a date picked up front.
- **Alternative kernels — not chosen.** Other commercial / open-source B-rep kernels exist but don't share the ACIS / ASM heritage that gives us lossless fidelity on AutoCAD-origin DWG 3D solids — translating AutoCAD-origin SAT blobs through any non-ACIS kernel introduces drift on edges, fillets, and tolerances that our migration-focused customers will see and reject. The KAL keeps a kernel swap technically possible at ~6–12 weeks of focused work if business conditions ever require it; it is not a hedge against ACIS being the right call today.

11. Industry adoption — who else picked these and why

For each component of the planned stack: notable adopters, why they picked it, known pain points in shipped products, and the resulting verdict for ActCAD.

11.1 ODA SDKs (Drawings, Visualize, IFC, BIM, MCAD, inWEB) — direct membership

- **Adopters.** Bricsys / BricsCAD (founding ODA member, the deepest user — Drawings + Visualize + BIM + IFC SDK), GstarCAD, ZWCAD, NanoCAD (all direct on ODA Drawings). Graphisoft ARCHICAD and Vectorworks use ODA Drawings as their DWG import / export layer. Bentley MicroStation, Trimble, Dassault hold memberships for format interoper. ARES Graebert wrote their own DWG library and uses

ODA selectively. **IntelliCAD itself swapped its in-house DWG core for ODA technology under the hood years ago**, so all IntelliCAD-derived products (ActCAD, progeCAD, CMS IntelliCAD) inherit ODA indirectly today.

- **Why.** Format coverage no one else ships under a single C++ API (DWG back to R12, DGN, IFC 2x3/4/4.3, Revit, Navisworks); aggressive Autodesk-version tracking; one membership covers everything; Visualize lets members drop expensive third-party graphics middleware (HOOPS, etc.).
- **Known pain.** Annual membership fee is non-trivial (Sustaining tier required for MCAD). API surface is wide but uneven — Revit *read* is mature, Revit *write* is partial. Threading model is heavy C++. Riding ODA's release cadence whether your QA is ready or not. Visualize scene graph is its own world — interop with engine-side picking / snapping is your problem.
- **Cost (vendor-published).** Limited Commercial \$3K first yr / \$2.25K renewal (100-seat cap). **Pro / Sustaining \$7.5K first yr / \$4.5K renewal** — unlimited commercial seats + Web/SaaS (inWEB) redistribution rights, **no per-seat royalty**. Enterprise / Founding \$37.5K first yr / \$18K renewal — adds full source + Git access + board nomination. Extensions priced separately (BimRv / BimNw / MCAD / Civil typically \$5K–\$10K/yr each on top of the membership). See §15 for the rollout. ([ODA pricing](#))
- **Verdict. Good fit, go direct.** ActCAD already pays ODA transitively; cutting to direct membership removes ITC as middleman and matches BricsCAD / GstarCAD / ZWCAD / NanoCAD.
- Sources: [ODA Members](#), [Bricsys ODA showcase](#), [IntelliCAD on ODA](#).

11.2 ACIS (Spatial / Dassault) — 3D kernel

- **Adopters.** Used by AutoCAD (via its ASM fork — see lineage note below), SpaceClaim (now ANSYS Discovery), Bentley MicroStation (uses both ACIS and Parasolid in different products), Hexagon products, BobCAD-CAM, IronCAD, CADopia, and **ActCAD itself today (via the IntelliCAD + ACIS stack)**. ACIS has been continuously productized since 1989 and is the second-most-deployed commercial B-rep kernel after Parasolid.
- **The AutoCAD lineage that matters.** AutoCAD shipped with ACIS for 3D solids starting with R13 (1994). Around 2001, Autodesk forked the ACIS source they had licensed and created **ASM (Autodesk Shape Manager)**, which is what AutoCAD, Inventor, Revit, and Fusion use today. The DWG format still stores 3D solids in the **ACIS SAT (Standard ACIS Text) blob format** as `AcDb3dSolid` entities — that's the on-disk representation. Reading a DWG with 3D solids gives you ACIS SAT. **Editing those solids with ACIS itself is the highest-fidelity path possible** outside of using ASM directly (which Autodesk does not license externally). Translating SAT into any non-ACIS kernel and back introduces drift on edges, fillets, tolerances, and history — drift that AutoCAD users see and reject.
- **Why this is the right choice for ActCAD.** Three reasons, in priority order: (1) **AutoCAD-fidelity migration story** — our customers are AutoCAD-DWG-native and that's our value prop; (2) **continuity** — ActCAD has been running on ACIS via IntelliCAD for years, so customer files, plugins, and workflows already assume ACIS semantics; (3) **maturity** — ACIS booleans, fillets, healing, and defeaturing are production-proven on the exact AEC + light-MCAD workloads ActCAD serves.
- **Known pain.** Largely single-threaded inside one body — concurrency happens across bodies, not within one operation. Module sprawl: base ACIS doesn't include robust booleans (Local Operations), import-healing (Healing), defeaturing (Defeating), or polyhedral / mesh (Polyhedral) — each is a separately-priced component. **No public production WASM build** — browser shell needs a separate

path (ODA's read-only translator + viewer in Phase 1, full ACIS-in-WASM is a custom engineering conversation if needed in Phase 2/3). Memory model uses derivation from `ENTITY` with explicit `lose()` semantics; KAL must encapsulate this so it doesn't leak into the rest of the engine.

- **Cost.** Commercial OEM — opaque, procurement-gated. Industry-estimate shape: **six-figure initial license + 15–20% annual maintenance + per-seat or per-deployment royalty + per-module component fees + DELA distribution restrictions**. See §15.1 for the full structure and §9 spike item 1b for the procurement workstream that returns real numbers. **All ACIS line items in §15.2 envelopes are procurement placeholders pending the spike's term sheet.**
- **Verdict.** Locked as the 3D kernel choice, contract terms gated on §9 spike items 1 and 1b. The technical reasoning (AutoCAD ASM lineage → lossless DWG fidelity) is decision-forcing; only the commercial terms remain to be closed.
- Sources: [Spatial ACIS](#), [ASM history](#), [AcDb3dSolid / ACIS in DWG](#), [ACIS components list](#).

11.3 ODA MCAD SDK — translator, not kernel

- **Adopters. Effectively none in production yet.** SolidWorks read opened June 2025; Inventor read followed; CATIA / NX / Creo / JT / Parasolid / Solid Edge are on the 2026–2027 roadmap. No shipped end-user CAD product is on it as a primary kernel.
- **Why members are interested.** Flat per-company pricing (no per-developer / per-seat), bundled into ODA membership extension — orders of magnitude cheaper than Parasolid (six-figures/yr + royalties) or ACIS. The only credible CATIA / SOLIDWORKS *write* path not controlled by Dassault.
- **Comparison.** ACIS (our chosen kernel, §11.2) is the production B-rep modeler; ODA MCAD is positioned as **translation / interop** — read SolidWorks / Inventor / CATIA, feed the resulting geometry into ACIS, edit there. The two complement each other and we use both.
- **Cost.** Bundled into ODA membership (Sustaining + extension, ~\$5K–\$10K/yr on top of the membership base). No per-seat royalty. **Parasolid for comparison: OEM only, fully opaque pricing, industry estimate six-figure upfront + per-seat royalty + annual maintenance** (Engineering.com / PROLIM analysts characterize CAD-component spend at 15–17% of ISV revenue once kernel + interop + visualization are stacked).
- **Verdict. Mismatch as a modeling kernel.** Use it as a **SolidWorks / Inventor / CATIA importer** in Phase 2-3 once it's mature. Do not stake the 3D pipeline on it.
- Sources: [ODA MCAD product](#), [MCAD SDK for SolidWorks](#).

11.4 Qt 6 — desktop shell

- **Adopters in CAD / DCC.** Autodesk Maya (Qt since 2011), MotionBuilder, Mudbox, parts of 3ds Max; The Foundry Nuke / Mari / Katana; SideFX Houdini; Foundry Modo; CATIA V6 / 3DEXPERIENCE; Allplan / Nemetschek; FreeCAD (Qt 5 → 6 in progress); QCAD; OpenSCAD. **Counter-example.** Blender (custom OpenGL UI).
- **Why.** Cross-platform from a single codebase; mature OpenGL / Vulkan integration via QOpenGLWidget / QRh; native theming; Qt Quick / QML for modern panels; signal-slot maps cleanly onto large C++ codebases.

- **Known pain. Commercial cost.** Qt for Application Development Professional / Enterprise is ~\$4,000–\$6,000 per developer per year for desktop; Small Business tier (rev-gated, caps at ~\$250K) is €530/year/dev. Add ~30–60% for mobile / embedded / Qt-for-WebAssembly. LGPL requires dynamic linking — static-link needs the paid license. Qt 5 → 6 migration is painful at scale (FreeCAD's Qt 6 work has been 2+ years and still partial).
- **Cost (vendor-published, locked to commercial — see §10).** Standard Application Development Enterprise: **~\$3,948–\$4,660/yr/dev**, mobile / WebAssembly add-ons +30–60%. Small Business (€530/yr Pro, USD 618/yr Enterprise) **almost certainly not eligible at Jytra's revenue**; model standard pricing. LGPL is rejected: a closed-source commercial product can't static-link Qt under LGPL v3, can't ship a code-signed iOS app cleanly under it, and can't keep Qt modifications private. Vendr reports buyers commonly negotiate **15–35% below first quote** on multi-year contracts; §9 spike item 6 owns that conversation. (Qt pricing, Vendr)
- **Verdict. Good fit, with a decision-forcing legal call.** Industry-standard for this product category. The LGPL-vs-commercial call gets made in §9 spike item 6.
- Sources: [Maya Qt SDK](#), [Qt pricing](#), [Qt Small Business](#), [FreeCAD Qt 6 #6992](#).

11.5 C++ vs Rust for the engine core

- **Adopters (Rust). Fornjot** (Hanno Braun) — experimental B-rep kernel, explicitly “reliability over features,” no shipped product on it. **Truck** (RICOS-JP) — Rust B-rep kernel, compiles to WASM; used by **CADmium** (Matt Ferraro). **Zoo.dev / KittyCAD** — the most serious commercial Rust CAD play; Rust geometry engine on the server (Vulkan / Nvidia), React frontend, app shipping. **Figma** — C++ canvas, Rust used for multiplayer sync server and hot-path tooling. Pattern: Rust at the edges, not the kernel.
- **Why.** Memory safety without GC, fearless concurrency, excellent WASM toolchain, cargo, no header / macro hell. For a from-scratch solver, type-checkable correctness.
- **Known pain.** Hiring depth in CAD geometry is in C++ not Rust. No mature commercial B-rep kernel ships a Rust binding (ACIS, Parasolid, and ODA SDKs are all C++ first). C++ FFI to ACIS / ODA is non-trivial — you pay it on every API boundary, which is why a Rust-primary core would be a tax on every inner-loop call.
- **Cost.** C++ and Rust toolchains are \$0. **Primary IDE is VS Code (or Cursor) + clangd + CMake Tools + Qt extension pack + AI agent (Claude Code / Cursor / Copilot)** — that's where the AI-augmented dev workflow lives in 2026. **Visual Studio Enterprise reserved for 2-3 designated Windows-perf devs** (IntelliTrace, Concurrency Visualizer, advanced profiler) at \$5,999 first / \$2,569 renewal. JetBrains CLion (~€979/yr) available to taste as an alternative. AI dev tooling budgeted at **~\$50/dev/mo blended** (Cursor Business \$40/mo, Copilot Business \$19/mo, Claude Code usage on top).
- **Verdict. Mismatch for the kernel; selective fit at the edges.** Confirms §2.4 — Rust earns its place in `net`, `script` `host`, `agent` (MCP), and new geometry algorithms. C++20 stays primary.
- Sources: [Fornjot](#), [Truck](#), [CADmium](#), [Zoo modeling-app](#), [Figma WASM](#).

11.6 WebGPU + WebAssembly — browser CAD

- **Adopters. AutoCAD Web** — Autodesk transpiled a major part of AutoCAD's ~15M-line C++ via Emscripten to WASM; same engine as desktop. **Figma** — C++ → WASM via Emscripten, shipped WebGPU rendering in 2024. **Adobe Photoshop Web** — same pattern. **ODA Drawings inWEB /**

Visualize inWEB — entire ODA C++ stack compiled to WASM. **CADmium** — Rust → WASM, three.js / WebGL today. **Tinkercad** — WebGL, JS-heavy. **Onshape** — NOT a WASM story: Parasolid runs native on AWS, browser only renders triangles via WebGL.

- **WebGPU baseline as of May 2026.** Chrome (since 113, 2023), Edge, Safari 26 (Sept 2025 — macOS Tahoe, iOS 26, visionOS), Firefox 141+ on Windows (July 2025), Firefox 145 on Apple Silicon. Firefox on Linux / Android still in progress. Babylon.js shows ~10× rendering speedup vs WebGL; WebGPU is the only path to in-browser compute shaders.
- **Known pain.** SharedArrayBuffer needs COOP / COEP headers — breaks embedding in customer intranets and some SaaS hosts. wasm32 caps at 4GB — large DWG / Revit forces wasm64 (narrower support). Filesystem assumptions need virtualizing via MEMFS / IDBFS. Binary size: stripped CAD WASM is tens of MB (need streaming compilation, dlopen-emulation code-splitting, `-Oz`). Debugging is dramatically worse than native.
- **Cost.** WebGPU / WebAssembly / WebGL2 are royalty-free W3C / Khronos browser APIs. **\$0.** No conformance fee.
- **Verdict.** **Good fit for a Visualize-only browser companion now; the full-editor browser story is a multi-year program** matching AutoCAD Web's trajectory, not a Phase-1 launch deliverable.
- Sources: [AutoCAD WebAssembly InfoQ](#), [WebGPU baseline 2026](#), [Figma WebGPU](#), [Onshape architecture](#).

11.7 ODA inWEB (Drawings inWEB, Visualize inWEB)

- **Adopters.** Drawings inWEB SDK opened to ODA members October 2024 — **no major shipping CAD product is publicly known to be on it for primary editing yet.** Members are building CDE / viewer apps. Visualize inWEB (`@inweb/viewer-visualize` on npm) is further along; several ODA members use it as a HOOPS / 3D-PDF viewer replacement. ODA's own *ODA Viewer* and the **VisualizeJS** demo viewer are the reference implementations.
- **Why.** Drawings inWEB is a **WASM transpilation of the C++ Drawings SDK** — file-format coverage is at parity with desktop Drawings SDK by construction.
- **Known pain.** Performance on large DWG (>200MB, dense annotation) lags desktop on initial-load and pan / zoom; browser-tab memory ceilings bite earlier than native. What's NOT at parity: full constraint engine, full LISP / .NET, plot / publish round-trip, certain custom-object proxies.
- **Cost.** Web/SaaS redistribution rights for inWEB begin at **ODA Sustaining (\$7.5K first yr / \$4.5K renewal)** — no per-deployment / per-seat royalty on top. Structurally cheaper than the Autodesk RealDWG path for a web product.
- **Verdict.** **Good fit for viewer / markup web companion in Phase 1; risky as the sole platform for full editing today.** Aligns with our native-Windows-first phasing in §5.
- Sources: [inWEB landing](#), [Drawings inWEB SDK release](#), [@inweb/viewer-visualize npm](#).

11.8 CMake + Emscripten — C++ → WASM toolchain

- **Successful CAD-scale codebases on this exact toolchain.** AutoCAD Web (15M-line C++ → WASM via Emscripten — the headline case). Figma (3× initial load improvement when they moved off asm.js). Adobe Photoshop Web. ODA Drawings inWEB / Visualize inWEB. VTK / Kitware (ships documented Emscripten build with `VTK_WEBASSEMBLY_64_BIT` for >4GB models). CAD Exchanger Web.

- **Recurring failure modes.** Threading + SharedArrayBuffer requires COOP / COEP headers (breaks embeds, ads); 4GB pointer cap on wasm32 forces wasm64 for large models; filesystem virtualization (MEMFS / IDBFS) for every `fopen`; third-party C++ deps (Boost, ICU, OpenSSL) need patching for the Emscripten toolchain; binary size in the tens of MB needs streaming compilation + code-splitting; long compile / link cycles on kernel-sized code (hours for full rebuilds with LTO); GL / GLES → WebGL2 / WebGPU shader translation; debugging vastly worse than native.
- **Cost.** CMake (BSD-3), Emscripten (MIT / NCSA), Cargo (MIT / Apache-2.0), Vcpkg (MIT), Conan (MIT) — **all \$0**. Kitware sells optional CMake support starting ~\$2,500 pre-paid blocks; nice-to-have, not required.
- **Verdict. Good fit and well-trodden.** Every relevant precedent uses this toolchain; the failure modes are published and the playbook is public.
- Sources: [AutoCAD WebAssembly InfoQ](#), [Figma WebAssembly](#), [Emscripten pthreads](#), [VTK Emscripten](#).

11.9 Summary verdict matrix

Component	Verdict
11.1 ODA SDKs (direct membership)	Good fit — matches BricsCAD / GstarCAD / ZWCAD / NanoCAD; removes ITC middleman
11.2 ACIS B-rep kernel	Locked — AutoCAD ASM lineage = lossless DWG fidelity; commercial terms gated on \$9 spike item 1b
11.3 ODA MCAD SDK	Not a kernel — use as SolidWorks / Inventor / CATIA <i>importer</i> in Phase 2-3
11.4 Qt 6 desktop shell	Good fit — industry-standard; budget LGPL constraints or ~\$5K/dev/yr commercial
11.5 Rust selectively, not for kernel	Confirmed — Rust at the edges (<code>net</code> , <code>script</code> , <code>agent</code>); C++20 stays primary
11.6 WebGPU + WASM browser	Viewer now; full editor multi-year — AutoCAD Web's trajectory, not a launch deliverable
11.7 ODA inWEB	Good fit for viewer; risky as sole platform for full editing today
11.8 CMake + Emscripten	Good fit — the exact playbook AutoCAD Web / Figma / ODA inWEB followed

12. Collaboration architecture in detail

12.1 What the leaders actually shipped

- **Onshape (PTC)** — only mainstream parametric CAD with simultaneous multi-user editing of the same part. Built on an immutable, append-only operation history: every user action becomes a sequential change in the workspace, cloud is the single source of truth, no files. Branch-and-merge modeled on git (USPTO 10,691,844; also 10,002,150 and 10,503,721). Merge is feature-by-feature replay with conflict detection at the feature graph level. Works because parametric CAD has a semantic operation language (the feature tree).

- **ARES Kudo (Graebert)** — full DWG editor in browser + Automation server (“ARES Commander running remotely”). Public collaboration is View-Only Links, Comments, Markups, Version Compare — *not* concurrent co-editing in the Figma sense. “Multiple users in parallel by alternating editing and viewing sessions” with session handover. Closer to soft-locked session sharing with cloud sync on save. ARES 2027 (April 2026) adds the A3 AI agent.
- **AutoCAD Web** — fundamentally single-user, backed by Autodesk Docs. AutoCAD 2025.1 adds “Can Edit” share links + Activity Insights for audit + DWG comparison. Autodesk does *not* advertise concurrent geometry co-edit; they advertise serialized collaboration with rich audit and diff.
- **Vectorworks Cloud Services** — Cloud Document Reviewer (browser comment / measure / markup on shared sheet layers, returns to desktop). Concurrent geometry editing is via on-premises Project Sharing — file-based, not cloud-native.
- **Bricsys 24/7** — a CDE (document management + streaming viewer + workflow), not a co-editor.
- **Snaptrude** — AEC analog of Onshape, browser-based, real-time simultaneous editing without checkout. AI generation of LOD 300 models in 7–10 minutes (October 2025).
- **Figma** (canonical reference) — explicitly rejected both OT and pure CRDT. **Server-authoritative last-writer-wins per property.** Each document a tree-of-objects (HTML-DOM-like). Per-document server process over WebSockets. **Each field updates independently — concurrent edits to the same property produce A or B, never a merged value.** Fractional indexing (base-95 strings — average between siblings to insert) for ordering. Cycle-rejection for tree-reparent. Orphaned objects hidden client-side until reconciliation.

12.2 Why generic CRDTs lose on DWG

- DWG geometry is not a flat list — it’s a spatially indexed graph with cross-references (block definitions, xrefs, layers, dimstyles, viewports).
- Tombstone GC blows up with millions of entities (Yjs scales to ~100k ops cleanly, OOMs at the millions-of-entity scale typical for an MEP riser DWG; Automerge OOMs on large transaction replay).
- Intent-preserving merge for “move this wall” requires understanding spatial constraints that generic CRDTs don’t model.
- Recent academic work (“Geometry-Aware CRDTs”, MDPI 2025) introduces geometric vector clocks + minimum bounding rectangles to capture spatial dependencies — still academic, not production.
- **The industry has converged on server-authoritative with domain-specific ordering, not generic CRDT.**

12.3 ActCAD’s two-tier model — detailed

Tier 1 (geometry edits): serialized op-log over the engine’s mutation stream.

- Every command (LINE, MOVE, ERASE, HATCH-modify, BLOCK-insert) is already an undoable transaction in the host today — that’s the existing AcDb transaction. We promote that transaction to a network operation.
- Server linearizes all geometry ops. Optimistic UI on the client, server can reject ops that violate referential integrity (door-to-wall, dim-to-entity, hatch-boundary-to-geometry).
- The `db` op-stream defined in §3 is exactly the wire format — same stream serves invalidation, undo, replication, agent context, and audit.

- Presence + selection locks at the UI layer prevent dual edits before they become conflicts.
- Branch / version semantics (Onshape-style) deferred to Phase 3 — SMB drafters won't think in git branches; presence + soft-lock is the right Phase-2 UX.

Tier 2 (annotations, comments, markups, layer properties, sheet titles, etc.): Figma-style LWW per property.

- Each field updates independently; concurrent edits to the same field resolve to A-or-B, never a merge.
- Fractional indexing for any z-order / draw-order semantics.
- Cycle-rejection for reparenting tree ops (sheet → layout → viewport).
- This is also the right place for Yjs / Automerge if we ever want one — bounded scope, text-and-JSON shaped data, no spatial constraints.

12.4 What we copy, what we avoid

- **Copy** from Onshape: serialized op-log with cloud as single source of truth; explicit version model (snapshot-and-restore) even if we don't expose branch / merge initially.
- **Copy** from Figma: LWW per property; fractional indexing; cycle-rejection for tree ops; orphan-and-reconcile for clients out of sync.
- **Avoid** Onshape's "every user is a software engineer" branch model — SMB AEC users want soft-lock + presence + LWW on properties, not git workflows.
- **Avoid** ARES Kudo's soft-lock-only ceiling — go server-authoritative op-log on geometry from day 1 to leapfrog them.
- **Avoid** generic Yjs / Automerge for geometry — confine to non-spatial metadata where it's bounded.

12.5 Phasing for collaboration

- **Phase 1** — the `db` op-stream is the wire format from the first commit. No co-edit yet; the browser shell is viewer + markup; markups go through Tier 2 (LWW).
- **Phase 2** — Tier 1 turns on for 2D drawings with the cloud document service. Real-time co-edit ships with presence + soft-lock + LWW annotations.
- **Phase 3** — Tier 1 extends to 3D / BIM; explicit branch / snapshot exposed in the UI for what-if workflows.

Sources: Figma multiplayer, USPTO 10691844 (Onshape branch / merge), ARES 2027, AutoCAD 2025.1 collaboration, Vectorworks Cloud, Bricsys 24/7, Geometry-aware CRDTs (MDPI 2025), CRDTs go brrr (Joseph Gentle).

13. Extensions and marketplace strategy

13.1 What the leaders actually shipped

- **Autodesk App Store + Autodesk Platform Services (APS, formerly Forge).** App Store accepts desktop add-ons (ObjectARX / .NET / LISP) and web apps using APS REST APIs. Initial review 24 hours, in-depth 2–3 days. Required artifacts: title, type, OS list, APS Client ID, screenshots, video,

privacy policy URL (embedded in description AND inside the app itself). Monetization is publisher-controlled. **“Security review” is functional QA, not sandboxing-and-capabilities** — native plugins run with full process privileges.

- **Onshape App Store / Extensions.** The cleanest web-extension model in the industry because designed cloud-native from day one. **Two extension types only:** (1) UI extensions = an HTTPS page rendered in an iframe inside Onshape, (2) Action extensions = REST calls to the developer’s server from Onshape’s UI (context menu / toolbar). OAuth2 — Onshape is the client, the app is the server. Parameterized URLs inject documentId / workspaceId / userId / companyId. 180-second timeout on GET / POST actions. **No third-party code ever runs inside Onshape’s process.**
- **BricsCAD Application Store + BRX.** BRX is BricsCAD Runtime eXtension — C / C++ and .NET API 100% source-compatible with AutoCAD ObjectARX. The value prop is “single code stream that runs on both BricsCAD and AutoCAD.” SDK free, dev enrollment free. No web analog because BricsCAD has no web product to speak of.
- **SketchUp Extension Warehouse.** Ruby. Every extension shares the same Ruby VM. Sandboxing is by convention, enforced by RuboCop-SketchUp linting (no global variables, no monkey-patching, no `$LOAD_PATH` mutation, one root module per extension). Manual review + automated lint at submission. Encryption (.rbe) supports paid extensions.
- **Vectorworks Marketplace.** Binary trust model (partner vs unknown) with no code signing or sandboxing (per Vectorworks 2026 docs). Unknown-developer plugins disabled by default with a session warning. Security theater — users click “enable anyway” universally.
- **SolidWorks Solution Partner Program. No app store at all.** Mandatory 10% revenue share + sales disclosure to Dassault — publicly criticized by partners as imposing requirements without providing a storefront.
- **AutoCAD Web “extensions.” LISP files only.** .lsp source uploaded to cloud profile. No .fas / .vlx compiled, no ARX, no .NET. ActiveX, 3D mesh / surface / solid creation, and most 3D-object modification are not available. Feature gap, not an architecture.

13.2 Architecture patterns

Two opposing models in production:

1. **In-process native plugins** (ObjectARX, BRX, Vectorworks SDK, SketchUp Ruby). Full power, no sandbox, distribution via store-as-shopfront. Security via developer reputation and manual review. Plugin-version-parity across host versions is the developer’s problem.
2. **Out-of-process cloud apps** (Onshape, APS). iframe UI + OAuth REST. Sandboxed by the browser. Capability-scoped by OAuth scopes. No code runs inside the host process.

Plugin parity across native and web does not exist in any shipped product. AutoCAD Web is LISP-only with zero ARX / .NET. Onshape has no native at all. BricsCAD has BRX (native) and nothing web. The Onshape model gives you real security review + capability scoping, but it can only express what the REST API exposes — you cannot extend the geometry kernel from inside.

13.3 ActCAD's extension model

Native desktop — keep BRX / ObjectARX compatibility. SMB AEC switching from AutoCAD is ActCAD's value prop; native plugins are table stakes. Provide:

- **BRX-compatible API surface** — match BricsCAD's "single code stream that runs on AutoCAD" pledge.
- **AutoLISP compatibility shim** — covered in §2.5 and §5.
- **Authenticode-style code signing** + "verified publisher" badge in the store (Vectorworks-style binary trust is rejected).
- **Automated pre-publication scanning** (call-graph, suspicious API use, network egress detection) before manual review.

Web — adopt the Onshape model. Two extension types only:

- **UI extension** = an HTTPS page in an iframe inside the ActCAD web shell.
- **Action extension** = REST call to a developer-hosted server, OAuth2 scoped.

Define a stable extension manifest declaring capability scopes (read-drawing, write-drawing, network-egress, file-system, user-presence, plot, agent-tool-publish) — start with maybe six scopes, expand only when needed.

Plugin parity = shared RPC surface, not shared binaries. The `cmd` module in §3 defines an internal command / query API. Native plugins call it in-process (BRX / .NET). Web extensions call the same API via OAuth REST. Same scopes, same semantics, same audit trail. This is the only credible path to "write once, run on web and desktop" — and even then, only for plugins that don't need raw geometry-kernel access.

13.4 Marketplace policy

- Revenue share: tiered, not flat. Free for the first \$5K/year of an extension's revenue, then 15% (Apple-equivalent for small business) to align with developer norms. Avoid the SolidWorks anti-pattern.
- Signing: required for both native and web.
- Categories: utilities, blocks / symbols, vertical packs (Architecture / MEP / Electrical / GIS), AI tools, integrations. AI-tool category needs a separate review track (see §14).
- Distribution: in-app store + web portal. Both list the same extensions.
- Agent-tool publishing: any extension that exposes MCP tools to the ActCAD agent runtime declares it in the manifest and goes through the AI-tool review track.

13.5 What to avoid (concrete anti-patterns)

- SolidWorks-style mandatory revenue share with no storefront.
- SketchUp's shared-VM-by-convention sandboxing — fine for SketchUp's installed base, dangerous for our new runtimes.
- Vectorworks-style binary trust — explicit capability scoping from day one, even if the initial buckets are coarse.
- Promising "write once, run everywhere" plugin parity — promise *API parity*, not *binary parity*.

Sources: Autodesk App Store Getting Started, Onshape Extensions, Onshape App Store, BRX API, SketchUp Extension Requirements, RuboCop-SketchUp, Vectorworks plug-in security, SolidWorks revenue share, AutoCAD Web LISP, Figma plugin system.

14. AI and agent strategy in detail

14.1 What the leaders actually shipped

- **Autodesk** — “Autodesk AI” brand across products + public MCP servers as the agent substrate.
Shipped:
- **AutoCAD Smart Blocks** — Search-and-Convert (BCONVERT) finds geometry matching an existing block; 2026 adds text-to-block-attribute recognition. Detect-and-Convert (Tech Preview) scans the whole drawing and suggests block candidates.
- **Markup Assist / Markup Import** — PDF / image markups → AI recognizes text commands (MOVE, COPY, DELETE) → proposes DWG edits.
- **Activity Insights** — audit log + diff (not generative).
- **Project Bernini** → “**Neural CAD**” — Bernini was the research model (10M shapes, ~3B params, text / sketch / image → 3D); now rebranded “Neural CAD” with the claim of editable CAD geometry in Forma / Fusion. No shipped product proves the editability claim yet.
- **Forma (was Spacemaker)** — Site Automation: generative site design exploration with multi-objective evaluation.
- **Autodesk Assistant + MCP** — **Revit 2027 ships a public MCP server (Tech Preview)** exposing six tool groups (model queries, sheet management, room management, schedules, exports, element ops): read everything, write a constrained set. **Fusion 2026 ships two MCP servers:** local Autodesk Fusion MCP (requires Fusion running, modeling + command execution) and remote Autodesk Fusion Data MCP (cloud, no Fusion required, data query). Both compatible with Claude Desktop / Cursor / VS Code / any MCP HTTP client.
- **BricsCAD AI** — AI Predict (ML-based ribbon command suggestion), AI Assistant (V26.2.03), Blockify (repetition detection for file-size reduction), Drawing Health (geometry cleanup), BIMIFY (AI classification of free-form 3D → IFC elements), auto-parametrize. **No public MCP server yet.**
- **Graebert ARES 2027** — A3 AI agent (built on OpenAI APIs) that selects / creates / modifies DWG entities; voice input in Kudo Professional; AI block generation; AI MText assistant. Deepest “agent in the inner loop” surface shipped on a DWG editor in 2026.
- **Adobe Firefly in Illustrator** — Generative Recolor (text-prompt palette generation across selected vector art) and Text-to-Vector Graphic (.svg output, then editable). Adjacent pattern: **AI outputs are first-class editable vector primitives, not raster overlays.** Trained on licensed Adobe Stock + public domain.
- **AI-native AEC startups.** Snaptrude — RFP → LOD 300 model in 7–10 min (adjacencies / zoning / codes / climate). Hypar — pivoted from text-to-BIM (Python / C# → IFC) to “suggestions” (Hypar 2.0, Jan 2025) focused on space planning, automatic editable masses / grids / columns / furniture from spatial programs. They publicly admitted text-to-BIM via chat alone was the wrong abstraction. Magicplan — mobile AR floor-plan capture.

14.2 The two architectural patterns

1. **AI as a tool** — inline, scoped, deterministic. Smart Blocks, Markup Assist, BIMIFY, AI Predict, Firefly Recolor. User invokes a specific AI-assisted command. AI suggests, user accepts. Undo is normal command undo. No persistent agent, no chat surface inside the canvas. **This is what's actually shipping and working.**
2. **AI in the inner loop** — agent + chat panel + MCP. Autodesk Assistant in Revit 2027, ARES A3, Fusion MCP. Chat panel converts NL → host command sequence. MCP is the architectural commitment — any LLM client can drive the host. Host exposes a curated tool surface. **Tech Preview status everywhere; no shipping CAD MCP server has documented transaction / undo semantics publicly.**

The closest validated pattern for “transactional, undoable, permission-scoped command surface” is actually **Figma plugins**: every plugin op is a Figma undo transaction; plugins must explicitly support undo; plugins run in QuickJS (compiled to WASM) inside an iframe sandbox; permissions declared in the manifest (currentuser, activeusers, fileusers, payments, teamlibrary). Not an MCP, but the cleanest shipped example of the contract.

14.3 Where the current generation goes wrong

- **Bernini-style generative CAD produces non-editable geometry.** That's why Hypar pivoted — chat-to-BIM gives a model that looks right but isn't tied to a parametric history a human can edit later. Autodesk's “Neural CAD” claim is unproven in product.
- **MCP without transactional semantics produces unfixable agent mistakes.** If an agent bulk-edits 5,000 wall parameters and corrupts the schedule, undo has to undo all 5,000 as one transaction.
- **Chat-as-primary-UI loses spatial context.** Hypar's lesson: “it's very hard to design anything real through a chat prompt.” Spatial editing wants spatial UX.
- **Activity Insights is not AI.** Autodesk lumps audit logging into the AI brand — don't confuse marketing taxonomy with shipped intelligence.

14.4 ActCAD's three-step strategy

Step 1 — Ship “AI as a tool” first (Phase 1). Three commands that win adoption fast and have clear scope:

- **Markup Assist analog** — PDF redline → DWG edit proposals.
- **Smart Blocks analog** — block detection across drawings (BCONVERT-equivalent).
- **Drawing Health** — layer / standards cleanup, lineweight normalization, duplicate-entity removal.

Each command produces deterministic, undoable suggestions. No chat. No MCP yet. This is the floor every shipping competitor is already at; we have to match it before we can talk about anything more ambitious.

Step 2 — Ship the MCP server as the integration surface, not the primary user UI (Phase 1–2). Mirror Fusion 2026's split:

- **Local MCP server** — requires ActCAD running, executes commands inside the host transaction, full read + write.
- **Remote MCP data server** — cloud, no host required, read-only DWG metadata / sheet / layer / block queries.

Tool groups along Revit 2027 lines: entity queries, layer / style management, block ops, sheet / layout management, plot / export, drawing health. Compatible with Claude Desktop / Cursor / VS Code / any MCP HTTP client.

The single most important architectural commitment: every MCP tool call is exactly one transaction in the host's existing undo stack. No public CAD MCP has documented this. Making it explicit — in the docs, in marketing, in the developer portal — is the differentiator that compounds. Permission scoping at the tool level (read-only / write / plot / file-system-egress) is declared in the MCP server's tool descriptions.

Step 3 — Conditional generative-CAD (Phase 3). Only ship “from prompt to drawing” features when editable round-tripping through the parametric / feature system is proven. Until then, stay on AI-suggestions-as-editable-primitives (Firefly pattern: vectors out, editable). This is the lesson Hypar learned the expensive way; we should learn it for free.

14.5 Agent tool design rules

- Every tool maps to one `cmd` invocation in the host.
- Every tool call = one undo group.
- Every tool declares its scope (read / write / file-egress / network-egress) and its blast radius (single-entity / drawing-wide / cross-drawing).
- Dry-run is supported for every write tool — agent can preview without commit.
- Rate-limited per session and per scope.
- Audit log retained server-side for every tool call (input, output, transaction id, scopes used) — this is also the training data for the next iteration.
- Tool catalog is versioned; clients negotiate capabilities on connect.

14.6 What we avoid

- The Bernini hype path — don't promise generative CAD geometry until editable parametric round-tripping is proven.
- Chat-in-canvas as the primary AI UX (Hypar pivoted for this reason).
- Over-indexing on a single LLM vendor — MCP exists exactly to keep that open.
- Shipping an MCP server without the single-undo-transaction commitment.
- Confusing audit / diff features with “AI” in customer-facing positioning.

Sources: [AutoCAD AI features](#), [AutoCAD 2026](#), [Project Bernini](#), [Autodesk MCP Servers](#), [Fusion MCP announcement](#), [Revit 2027 MCP Tech Preview](#), [Revit 2027 MCP analysis](#), [ARES 2027 A3](#), [BricsCAD AI](#), [Adobe Firefly Vector Recolor](#), [Snaptrude AI](#), [Hypar 2.0](#), [Hypar text-to-BIM lesson](#), [Snyk MCP CAD list](#), [Figma plugin docs](#).

15. Cost of usage — third-party stack, shells, tools

Per-component cost data appears inline in each §11 subsection. This section rolls those up into a single reference table and three modeled annual budget envelopes at typical Phase-1 / Phase-2 / Phase-3 team sizes.

All dollar figures are vendor-published or reported secondary-source estimates as of late 2025 / early 2026. Treat any opaque-OEM number as a directional placeholder — they need a live procurement conversation, not a guess.

15.1 Unified pricing reference

Component	License model	Public price	Per-seat / royalty	Source
ODA — Limited Commercial	Annual membership	\$3K first yr / \$2.25K renewal	None — but 100-seat cap	opendesign.com/pricing
ODA — Pro / Sustaining	Annual membership	\$7.5K first / \$4.5K renewal	None — unlimited + Web/SaaS rights	opendesign.com/pricing
ODA — Enterprise / Founding	Annual membership	\$37.5K first / \$18K renewal	None — adds source code + Git	opendesign.com/pricing
ODA extensions (BimRv / BimNw / MCAD / Civil / Scan-to-BIM)	Add-on, Sustaining+	\$5K–\$10K/yr each (reported)	None	ODA FAQ
Spatial / Dassault ACIS — Initial OEM license	One-time OEM fee	Opaque — industry estimate \$150K–\$300K, procurement-gated	One-time, gates access to headers + libs	Spatial ACIS
ACIS — Annual maintenance	% of license fee	15–20% of initial fee (~\$30K–\$60K/yr typical)	Required for updates, hotfixes, CTC access	Spatial support
ACIS — Per-deployment royalty (<i>primary ask</i>) / per-seat (<i>fallback only</i>)	Volume-tiered	Opaque, tiered	Per-deployment is the locked negotiating ask — ~10× cheaper than per-seat over 3 years at ActCAD scale. Full comparison math, gotchas, and negotiating sequence in §15.1.1 .	—
ACIS — Component modules (Local Ops, Healing, Defeaturing, Polyhedral, AGM, HLR, Lop)	Each priced separately on top of base	Opaque, per-module	A “complete” ACIS for our scope typically needs 3–5 modules; scope module list before signing	ACIS components
ACIS — CTC support hours	Bundled + overage	~20–40 hr bundled; \$300–\$500/hr beyond (estimate)	Engineering-level support from Spatial's Corporate Technical Consulting	—
ACIS — Source escrow (<i>optional, recommended</i>)	Annual	~\$5K–\$15K/yr typical	Business-continuation insurance if Spatial discontinues	—
Siemens Parasolid (<i>not chosen</i>)	OEM	Opaque	n/a	Parasolid

Component	License model	Public price	Per-seat / royalty	Source
Qt for Small Business — App Dev	Sub, per-dev	€530/yr Pro, USD 618/yr Ent	n/a — but ≤€1M revenue cap, max 3 licenses	Qt SBE
Qt — Application Development (standard)	Sub, per-dev	~\$3,624/yr Pro, \$3,948–\$4,660/yr Ent	n/a	Qt pricing
Qt — Device Creation / Mobile / WASM add-ons	Sub, per-dev	+30–60% over App Dev	n/a	Qt pricing
Qt LGPL	LGPL v3	\$0	n/a — but no static linking, no signed iOS	Qt LGPL
CMake / Emscripten / Cargo / Vcpkg / Conan	OSS	\$0	None	—
Kitware CMake support (optional)	Pre-paid	\$2,500 / 12 mo	None	Kitware support
WebGPU / WebAssembly / WebGL2	W3C / Khronos open	\$0	None	—
Tauri (if used for launcher app)	MIT / Apache-2.0	\$0	None	—
Electron (rejected for editor)	MIT	\$0	None	—
.NET 8 / NativeAOT	MIT	\$0	None	.NET free
MCP protocol + SDKs	MIT, donated to Linux Foundation	\$0	None	MCP
AutoLISP runtime	No commercial library exists — build in-house	\$0 (ECL embedded LGPL viable starting point)	None	AutoLISP history
Clerk Auth	Per-MAU, vendor-published	Free ≤10K MAU; \$25/mo base + \$0.02/MAU	n/a	Clerk pricing
Auth0	Per-MAU	\$150/mo for 500 MAU; escalates fast	n/a	Auth0 pricing

Component	License model	Public price	Per-seat / royalty	Source
		— enterprise \$30K+/yr		
Stripe Payments	% of volume	2.9% + \$0.30 domestic card	n/a	Stripe pricing
Stripe Billing	% of volume	0.7% of billing volume	n/a	Stripe Billing
AWS GPU — g4dn.xlarge (T4)	On-demand	\$0.526/hr (\$4,608/yr 24×7)	n/a	AWS GPU
AWS GPU — g5.xlarge (A10G)	On-demand	\$0.916/hr (\$8,024/yr 24×7)	n/a	AWS G5
JetBrains All Products Pack	Sub, per-dev	€979/yr (~\$1,050)	n/a — yr-2 ~20% off, yr-3 ~40% off	JetBrains
Visual Studio Enterprise (2-3 perf devs only)	Sub, per-dev	\$5,999 first / \$2,569 renewal	n/a — reserved for Windows deep-profiling work	VS pricing
VS Code + clangd + Qt extensions (primary IDE)	OSS	\$0	n/a	VS Code
AI dev tooling (Claude Code / Cursor / Copilot blended)	Per-dev sub	~\$50/dev/mo (\$600/yr); Cursor Business \$40, Copilot Business \$19, Claude Code usage on top	n/a	Cursor pricing , Copilot Business
GitHub Enterprise Cloud	Sub, per-dev	\$21/dev/mo (\$252/yr)	n/a — 50-100 seat min on annual	GitHub pricing
GitHub Advanced Security	Sub, per-dev	+\$19/dev/mo (\$228/yr)	n/a	GitHub pricing
GitHub Actions (standard)	Per-minute	\$0.006/min (2-core Linux, effective Jan 2026); 50K min/mo bundled in Enterprise Cloud	n/a	Actions pricing

15.1.1 ACIS royalty model — per-deployment vs per-seat

The single largest variance in the ACIS commercial envelope is the **royalty model** (line item 3 in §15.1's cost-structure table). The two models can differ by ~10× over a 3-year window for an SMB-volume / low-price product like ActCAD. This subsection is the negotiating reference for Spike 1b.

The two structures, plainly:

	Per-seat royalty	Per-deployment royalty
What triggers payment	Each active licensed user, recurring (annual)	Each unique install, typically one-time
Scales with	Total customer base × time	New installs per year
Risk shape	Linear forever; punishes long-lived perpetual customers	Front-loaded; amortizes over seat lifetime
Friendly to	Low-volume / high-price MCAD products (Plasticity, Shapr3D, KeyShot)	High-volume / low-price SMB CAD products (ActCAD, BricsCAD-class)
Reporting overhead	Per-customer active seat counts each quarter	Install events with unique deployment IDs

Illustrative math at ActCAD scale. Hypothetical \$50 royalty unit, 30,000 installed seats today, 3,000 new installs / year, 5-year average seat lifetime. Numbers are illustrative — actual rates come from Spatial's term sheet — but the relative shape holds:

Year	Per-seat (\$50/seat/yr)	Per-deployment (\$50/install, one-time)
Y1	$30,000 \times \$50 = \text{\$1.50M}$	$3,000 \times \$50 = \text{\$150K}$
Y2	$33,000 \times \$50 = \text{\$1.65M}$	$3,000 \times \$50 = \text{\$150K}$
Y3	$36,000 \times \$50 = \text{\$1.80M}$	$3,000 \times \$50 = \text{\$150K}$
3-yr total	\\$4.95M	\\$450K

~10× difference. This is why the Phase 2 / 3 royalty placeholders in §15.2 assume per-deployment. If Spike 1b returns a per-seat-only offer at comparable headline rates, the Phase 3 envelope grows by roughly \$1M+.

When per-seat actually wins (none of these describe ActCAD, but document them so the comparison is honest): 1. **High-price low-volume MCAD products.** Plasticity / Shapr3D / KeyShot — selling \$500–\$5K perpetuums at 5K–50K seats. A \$50/seat royalty is 1–10% of revenue and reporting is simpler. 2. **Pure subscription with high churn.** 40%+ annual churn — per-seat self-terminates with the lapsed seat; per-deployment is sunk cost regardless. 3. **Single-customer enterprise / site license.** One customer, hundreds of seats — Spatial usually offers a flat site fee that beats both row-level models.

Contract gotchas that change the actual cost (must be in writing before signing): - **Per-deployment “unique” definition.** Does reinstall on the same machine count? Hardware change? VM migration? OS reinstall? Vague language turns “one-time” into “recurring.” Push for *machine-fingerprint or install-ID-based* with reinstalls on same machine excluded. - **Per-seat “seat” definition.** Lapsed-subscription seats still counted for the quarter? Concurrent vs named users? Trial / evaluation seats counted? 30-day grace period for inactive seats? - **Volume tier breakpoints.** A \$50/seat headline might drop to \$20 above 50K seats. Per-deployment might have no tiers, or steep tiers. **Tier shape matters more than the headline rate.** - **Minimum annual royalty floor.** Most OEM contracts carry a minimum regardless of volume — sets the actual cost at low volumes. Push for \$25K–\$50K floor rather than \$100K+. - **Subscription vs perpetual treatment.** Some contracts charge differently for subscription vs perpetual customers. Ask explicitly. - **Audit + true-up cadence.** Quarterly reporting + annual true-up is standard; some contracts allow annual reporting which reduces our internal overhead (line item 9 in §15.1). - **Education / trial / internal-QA carve-outs.** Excluded from royalty count? Get it written.

The negotiating ask sequence for Spike 1b (ordered by priority): 1. **Per-deployment royalty, one-time**, with a precise unique-deployment definition (machine-fingerprint based, same-machine reinstalls excluded). 2. **Volume-tiered**, with breakpoints at 5K / 25K / 100K cumulative deployments and a meaningful tier decay (e.g., \$50 → \$30 → \$15 → \$5). 3. **Carve-outs:** trials, education, internal QA, evaluation copies all excluded from royalty count. 4. **Annual minimum floor** as low as possible — target \$25K–\$50K rather than \$100K+. 5. **Per-seat as a fallback only** if per-deployment isn’t on offer — and only with a steep volume curve (e.g., \$50 → \$5 above 50K seats) where 5-year amortized cost beats their per-deployment alternative.

The decision rule. Accept per-seat only if Spatial refuses per-deployment AND the 5-year amortized per-seat cost (using realistic seat-count + churn projections) is lower than the per-deployment alternative they would have offered. That comparison — both quotes side by side — is the negotiating leverage. **The plan locks in per-deployment as the default ask, with per-seat documented as an explicit fallback decision, not an accident.**

15.2 Modeled annual budget envelopes

Three scenarios sized to the engineering org in §5’s phasing. **These are modeled rollups, not committed numbers.** Assumptions: ODA Sustaining from year 1 (Web/SaaS rights for inWEB), Qt commercial App Dev Enterprise standard tier (Small Business eligibility unlikely at Jytra’s scale), **ACIS as the 3D kernel under a Spatial OEM contract** with the cost structure in §15.1 (initial fee + maintenance + royalty + module fees), **VS Code primary + Visual Studio Enterprise for 2-3 designated Windows-perf devs only**, AI dev tooling at ~\$50/dev/mo blended.

ACIS line items are procurement placeholders pending §9 spike item 1b’s term sheet. Industry-estimate values are used as directional anchors only — the actual numbers come from the NDA conversation with Spatial. See §15.3 for modeled-vs-opaque split.

Phase 1 — 8 engineers (months 0–12)

Line item	Year 1
ODA Sustaining (incl. inWEB rights)	\$7,500
ODA Revit add-on (BimRv Standard)	\$5,000
Qt commercial — App Dev Enterprise (8 devs × \$4,000)	\$32,000
ACIS — initial OEM license (<i>placeholder — opaque, \$9 spike 1b</i>)	\$200,000
ACIS — annual maintenance (<i>placeholder — 17.5% of license</i>)	\$35,000
ACIS — component modules (<i>placeholder — Local Ops + Healing + Defeaturing</i>)	\$50,000
ACIS — royalty (<i>Phase 1 is internal beta — minimal seats, placeholder</i>)	\$5,000
ACIS — source escrow (<i>recommended</i>)	\$10,000
Visual Studio Enterprise (2 perf devs × \$5,999)	\$11,998
VS Code + clangd + Qt extensions (rest of team)	\$0
AI dev tooling (8 devs × ~\$50/mo)	\$4,800
JetBrains All Products Pack (4 devs to taste × \$1,050)	\$4,200
GitHub Enterprise Cloud (8 × \$252)	\$2,016
GitHub Advanced Security (8 × \$228)	\$1,824
GitHub Actions large runners + cache	\$5,000
AWS dev infra (compute + storage + 1× g4dn dev box)	\$8,000
Auth / billing (nominal — Clerk free tier, Stripe test mode)	\$1,000
Phase 1 yr 1 stack + tools + cloud total	~\$383,300 (incl. ~\$300K ACIS one-time + recurring)

Phase 2 — 20 engineers (months 12–24, GA at month 24)

Line item	Year 2 (renewal-mode where applicable)
ODA Sustaining renewal	\$4,500
ODA extensions (BimRv + MCAD + Civil)	\$15,000
Qt commercial — App Dev Enterprise (20 × \$4,000)	\$80,000
Qt mobile / WASM add-ons (+40% on 5 web/mobile devs × \$4,000)	\$8,000
ACIS — annual maintenance (<i>recurring</i>)	\$35,000
ACIS — component modules renewal	\$50,000
ACIS — royalty (<i>GA at month 24 — placeholder, assume 2,000 deployments × \$50</i>)	\$100,000
ACIS — source escrow	\$10,000
ACIS — CTC support hours overage (<i>placeholder</i>)	\$15,000
Visual Studio Enterprise (3 perf devs × \$2,569 renewal)	\$7,707
AI dev tooling (20 devs × ~\$50/mo)	\$12,000
JetBrains All Products Pack (10 devs to taste × \$840 yr-2 disc.)	\$8,400
GitHub Enterprise Cloud (20 × \$252)	\$5,040
GitHub Advanced Security (20 × \$228)	\$4,560
GitHub Actions + build infra	\$15,000
AWS production cloud (compute + 4× g5 streaming + storage + egress)	\$60,000
Auth (Clerk B2B, ~25K MAU)	\$12,000
Stripe Billing (assume \$200K ARR × 0.7%)	\$1,400
Stripe Payments (assume \$200K × 2.9%)	\$5,800
Phase 2 stack + tools + cloud total	~\$449,400 (incl. ~\$210K ACIS recurring + royalty at GA)

Phase 3 — 30 engineers (months 24–36)

Line item	Year 3
ODA — upgrade to Founding (source access)	\$37,500 first / \$18K renewal
ODA extensions (full extension set)	\$30,000
Qt commercial (30 × \$4,000)	\$120,000
Qt mobile / iOS / Android / WASM add-ons (10 devs × \$4,000 × 50%)	\$20,000
ACIS — annual maintenance	\$35,000
ACIS — component modules renewal (<i>may add Polyhedral + AGM for full 3D + BIM</i>)	\$75,000
ACIS — royalty (<i>at scale — placeholder, 10,000 deployments × \$50</i>)	\$500,000
ACIS — source escrow	\$10,000
ACIS — CTC support + WASM custom engineering (<i>if mobile / web needs full ACIS</i>)	\$40,000
Visual Studio Enterprise (3 perf devs renewal)	\$7,707
AI dev tooling (30 devs × ~\$50/mo)	\$18,000
JetBrains All Products Pack (12 devs to taste × \$630 yr-3 disc.)	\$7,560
GitHub Enterprise Cloud (30 × \$252)	\$7,560
GitHub Advanced Security (30 × \$228)	\$6,840
GitHub Actions + build infra	\$30,000
AWS production (compute + 12× g5 streaming + CDN + egress at scale)	\$200,000
Auth (Clerk B2B, ~100K MAU)	\$30,000
Stripe Billing + Payments (assume \$2M ARR)	\$73,000
Phase 3 stack + tools + cloud total	~\$1,255,700 (incl. ~\$660K ACIS recurring + royalty at scale; royalty scales with deployments)

15.3 What's modeled vs what's opaque

- **Modeled (vendor-published):** ODA, Qt, all OSS tooling, AWS GPU, Clerk, Stripe, JetBrains, Visual Studio, GitHub.

- **Opaque (needs procurement conversation — placeholders pending §9 spike item 1b):** ACIS initial license + maintenance + per-module fees + royalty model + DELA, Auth0 enterprise, Microsoft Unified Support. **All ACIS numbers in the rollup are industry-estimate placeholders, not vendor quotes.**
- **Variable with revenue / users:** Stripe (% of volume), Clerk (per-MAU), AWS (per-instance-hr × utilization).

15.4 Commercial commitments and cost-driven decisions

ActCAD is a closed-source commercial product. The license tier on every component is chosen accordingly — paid where standard commercial practice (static linking, code-signing, private modifications, unlimited seats) requires it, free where the OSS license cleanly permits it for closed-source use.

1. **ODA Sustaining from day 1 — LOCKED.** \$7.5K first / \$4.5K/yr renewal for unlimited commercial seats + inWEB Web/SaaS redistribution rights with **no per-seat royalty** is the single best-value line item in the entire stack. Limited Commercial's 100-seat cap is a footgun at our scale; Founding (~\$37.5K) considered in Phase 3 for source + Git access + business-continuation rights.
2. **Qt 6 commercial — LOCKED.** Standard Application Development Enterprise per developer. LGPL is *not* viable for ActCAD: static linking is forbidden under LGPL v3, code-signed iOS distribution can't satisfy the user-relinking clause, and we need to keep our Qt modifications private. Small Business tier is almost certainly not available at Jytra's revenue. §9 spike item 6 negotiates the multi-year quote; target 15–35% below first quote per Vendr-reported norm.
3. **ACIS commercial OEM contract — LOCKED as the kernel choice; commercial terms OPEN pending Spike 1b.** Rationale: AutoCAD's 3D solids are stored as ASM (ACIS fork) SAT blobs in DWG; only ACIS round-trips them without drift. ActCAD already uses ACIS today through IntelliCAD; this decision puts the contract directly in our name and removes the IntelliCAD middleman. Cost shape is initial OEM license + 15–20% annual maintenance + royalty + per-module fees + DELA — see §15.1. **Royalty model — per-deployment is the LOCKED negotiating ask** (per-seat documented as fallback only; ~10× cost variance over 3 years at ActCAD scale — see §15.1.1 for the comparison math and full negotiating sequence). **All §15.2 ACIS line items are industry-estimate placeholders pending the term sheet from Spatial.** Once that lands, Phase-1 budget firms up to a real number.
4. **VS Code (or Cursor) primary, Visual Studio Enterprise reserved — LOCKED.** Primary IDE is **VS Code + clangd + CMake Tools + Qt extension pack + AI agent (Claude Code / Cursor / Copilot)** for the entire C++ team — that's where the AI-augmented dev workflow lives in 2026. Visual Studio Enterprise (~\$6K first / \$2.6K renewal) is **reserved for 2-3 designated Windows-perf devs** for IntelliTrace, Concurrency Visualizer, and advanced profiler work into ODA / Qt call stacks. JetBrains CLion available to taste as an alternative. Saves ~\$12K Phase 1, ~\$18K Phase 2, ~\$30K Phase 3 versus the original plan of buying VS Enterprise for the whole C++ team.
5. **AI-augmented team math.** Qt's commercial license is per-human-developer; AI agents are tools, not licensees. AI doesn't reduce the per-seat *rate*, it reduces the per-seat *count* — a 12-dev AI-augmented Phase-2 team can ship what 20 devs shipped pre-AI, so Qt's line drops from \$80K to ~\$48K via headcount, not via license loophole. Same logic applies to ODA seats (unlimited under Sustaining — no effect), GitHub / Visual Studio / JetBrains (per-human, scale with headcount). Budget for AI dev tooling itself at **~\$50/dev/mo blended** (Cursor Business \$40, Copilot Business \$19, Claude Code on top).

6. **All build / runtime tooling stays free.** CMake, Emscripten, Cargo, Vcpkg, Conan, WebGPU / WebAssembly, .NET 8 / NativeAOT, MCP, Tauri (launcher app only) are all permissive OSS — no commercial concern for our use.
7. **AutoLISP must be built in-house.** No commercial runtime exists to license; every IntelliCAD-class vendor rolls its own. Phase-1 LISP team is a real headcount commitment, not a vendor decision.
8. **Total stack + tools + cloud envelope** — roughly **~\$383K Phase 1, ~\$449K Phase 2, ~\$1.26M Phase 3** (Phase-3 royalty scales with deployment volume). The ACIS line is the single largest cost driver and the single largest source of variance — locking the term sheet in Spike 1b is what converts these from placeholders to commitments. Salary still dwarfs the stack cost at every phase, but the kernel-commercial conversation is a real one that procurement / legal own from Spike 1b onward.

16. Memory architecture — designing for headroom from day 1

Executive summary. Full engineering reference is in `docs/memory-architecture.md` (~900 lines: concurrency model, KAL contract, eviction protocol, failure modes, CI gates, anti-patterns, IntelliCAD-migration teaching, phase-by-phase rollout, glossary, references). Read this section for the strategic picture; read the companion document before writing code that touches `db`, `geom`, `render`, or any cache.

The CAD operating model is simple to state and hard to make fast: **open the drawing once, hold it entirely in memory, do all reads and writes against that in-memory state, write back to disk on save.** This is what AutoCAD has done since 1982 and what every serious DWG editor still does today. The model itself is correct and not up for debate. What turns it into a bottleneck is that “in memory” is doing more work than people think — a 250MB DWG can expand to several GB of working set once you add the kernel’s body graph, the tessellation cache, spatial indexes, the undo log, and the renderer’s GPU buffers. If the architecture doesn’t budget that memory and police access patterns from commit 1, you ship a product whose user reviews say *“moderately large drawings frequently get stuck”* — the exact reputation the current ActCAD inherited from IntelliCAD and the exact reputation this re-architecture exists to escape.

This section is the architectural commitment for how we avoid that. It is written in plain language by design: every engineer joining the team should be able to read it once and know what the rules are.

16.1 The five layers of memory in a live CAD session

People talk about “the drawing in memory” as if it were one thing. It’s really five layers, each with its own size, owner, and eviction policy. Get this picture in your head and the rest of this section follows.

Layer	What lives here	Typical size (250MB DWG with 3D solids)	Owner module	Evictable under pressure?
1. DWG database	Entities, handles, layers, blocks, xrefs, dimstyles, attributes — the user's actual work	400–800 MB	db	Never — single source of truth
2. ACIS kernel bodies	The ENTITY graph for every 3D solid the user has touched	200 MB – 2 GB	geom via KAL	Partial — lazy-loaded on first touch; attributes lazy beyond that
3. Tessellation cache	Triangles + GPU buffers for everything currently being drawn	500 MB – 4 GB	render	Yes — LRU evict; regenerate on demand
4. Spatial indexes	R-tree / BVH for hit-test, snap, frustum-cull, selection-window	50–200 MB	db + render	Rebuildable — recompute from db
5. Undo / op-log	Delta records for reversal + co-edit replication + agent context + audit	100–500 MB over a long session	db	Truncatable — drop oldest beyond threshold

The one rule: Layer 1 (db) is sacred and never evicted. Everything above it is a cache or a derived view, and the budget says how much memory we'll spend on each. When pressure hits, the eviction order is fixed: tessellation (3) first → spatial indexes (4) → lazy-loaded ACIS attributes (2). Layer 1 stays put. The user's work is never at risk from a memory-management decision.

16.2 Eight architectural commitments that keep responsiveness flat as drawings grow

These are the design rules, in priority order. Each prevents a specific class of bottleneck that other CAD products live with.

1. The db is the only owner of drawing state. Everything else is a cache.

If render needs triangles, render owns the triangle cache and db doesn't know about it. If snap needs an acceleration structure, snap rebuilds it from db and discards it. This is what makes Layer 1 / Layer 3–5 separation real instead of theoretical. The reason it matters in practice: when something is wrong in memory, you always know who to ask, and you can blow away every cache without losing the user's work. Single-source-of-truth is not a slogan; it's a recovery strategy.

2. One write path, many read paths, never blocked by each other.

db mutations are serialized through one transaction queue on a worker thread. Reads — render, hover, snap, selection, agent query, property panel — go against the committed state via a **copy-on-write snapshot**. **No reader ever waits on a writer**. This is the single most important commitment in this section. It's what makes the UI stay at 60fps while a long ACIS boolean is running, what makes the AI agent able to

query state during an edit without deadlocking, what makes co-edit possible without lock storms. Database people call this MVCC (multi-version concurrency control). It's the same pattern PostgreSQL uses. A CAD database is a database, and the rules that make databases fast apply here.

3. Spatial index on load, not on demand.

The moment a drawing finishes loading, the R-tree / BVH is built. Every “what's at point X” query — pick, hover, OSnap, frustum cull, selection window, agent “find all entities in this room” — goes through the index. **Linear scan over entities is not a public API.** This is the single decision that separates snappy from unusable on a 250k-entity drawing. ODA's spatial filter can be wrapped here; we don't reinvent it. If we ever profile a hot path and find a linear scan, that's a bug fix, not an optimization.

4. Lazy materialization of ACIS bodies.

ACIS bodies stay as opaque SAT blobs in `db` until the first geometric operation on that body. A drawing with 500 3D solids the user never queries shouldn't materialize 500 solids on open. ACIS supports this natively — we plumb it through the KAL. **Open time should be bounded by parse + index, not by kernel materialization.** When a user opens a 250MB DWG with 500 solids and clicks one of them, they wait for *one* solid to materialize, not 500.

5. Tessellation lives on the GPU and is evictable.

Once a body is tessellated, the triangles go to GPU memory and stay there for the render loop. Under CPU memory pressure, the LRU tessellation is evicted; on next view it's regenerated from the cached B-rep. The tessellation cache has a **hard cap** (default: 30% of process working set) so it can never starve `db`. This is what `render` exists to do, and it's why `render` is a separate module from `geom` — different concerns, different eviction policies, different owners.

6. Undo is a delta log, not a state snapshot.

Every undoable operation stores its inverse, not a copy of the world. Reversing `move(handle, dx, dy)` is `move(handle, -dx, -dy)` — bytes, not megabytes. Full-state snapshots are what blow heap in long sessions; deltas are what let AutoCAD survive an 8-hour edit session without the user noticing memory growth. This delta log is **the same op-stream** that drives sync, replication, agent context, and audit. One stream, five consumers (§3, §12). Building it any other way means writing the same plumbing five times.

7. The memory budget is declared, enforced, and visible.

The engine boots with a declared memory budget — e.g., “*2× the loaded DWG size, capped at process working set minus 1 GB headroom for the OS.*” Each cache layer has a sub-budget. When a sub-budget is exceeded, the layer evicts according to its policy. When the total is exceeded after eviction, the engine **fails loudly to the user** — “*this drawing exceeds your memory budget; please close other drawings or raise the limit in Preferences*” — instead of silently swapping to disk. **Silent swap is the worst possible failure mode:** the user thinks the app is frozen, force-quits, and loses work. Loud failure is recoverable; silent swap is a support ticket and a churn risk.

8. Long operations run on workers; the UI thread does input + render only.

Open, save, regen, plot, index build, ACIS boolean, fillet, hatch boundary — all on worker threads with progress + cancel. Main-thread budget per frame is **8 ms** (target 16 ms for 60fps with headroom). Frame budget asserts in debug builds: any UI-thread work over 8 ms triggers a debug break, caught at write time by the engineer, not by a customer support ticket. This is how you guarantee responsiveness — by making “slow on the UI thread” a build-breaking error from commit 1.

16.3 Where bottlenecks actually appear in shipping CAD products, and the architectural answer to each

Plain-language version of what “*moderately large drawings get stuck*” usually means in practice, and how the rules above address each symptom.

Symptom the user sees	What’s really happening	Architectural answer
“Opens slowly on big files”	Eager ACIS materialization; spatial index built lazily on first pick	Rule 4 (lazy ACIS) + Rule 3 (index on load) — open time bounded by parse + index, not kernel work
“Stutters when I pan / zoom”	UI thread doing render + invalidation + new tessellation in one frame	Rule 5 (GPU-resident, evictable tess) + Rule 8 (8 ms UI budget)
“Hover lags on dense drawings”	Linear scan for hit-test	Rule 3 — spatial index is mandatory, scan-over-entities is forbidden
“Hangs for a few seconds when I save”	Save serializes the whole DWG on the UI thread	Rule 8 — save is a worker task with progress bar
“Editing a big solid freezes the app”	ACIS boolean on the UI thread, no copy-on-write snapshot	Rule 2 (MVCC reads) + Rule 8 (worker) — view keeps rendering against the pre-edit snapshot while the worker computes
“Crashes after a long session”	Undo log grew unbounded; or tessellation cache grew unbounded	Rule 6 (delta log + truncation) + Rule 5 (cache cap) + Rule 7 (budget enforcement)
“Force-quit because it ‘froze’ (but it was paging)”	Silent swap to disk under memory pressure	Rule 7 — fail loudly, never silently swap
“Property panel takes a second to update”	Eager load of all entity attributes on selection	Rule 4 (lazy attributes) — narrow query, on-demand
“Plugins make the app slow”	Plugin code running on the UI thread with full db access	<code>cmd</code> is the only seam; plugins go through it; long-running tool calls forced to worker (§13, §14.5)

16.4 What to measure, and how to keep it from regressing

Plain rule: **the architecture above doesn't survive contact with the codebase unless CI measures it on every PR.** Specific gates we add at Phase-1 start:

1. **Open-time budget.** Corpus of 50 DWGs (small / medium / large / huge / pathological). PR fails if median open time regresses > 10%. Each drawing has a documented expected time.
2. **Frame-time P99 budget.** Synthetic “*pan-zoom-rotate-hover*” loop over each corpus drawing. P99 frame time logged per PR; regression past 16 ms = fail.
3. **Memory ceiling per drawing class.** Peak RSS during the synthetic loop. Hard ceiling per class; PR exceeding it fails.
4. **Allocation count regression.** Total `malloc` count in the open-edit-save loop. A sudden 2× spike usually means a new code path forgot to use the entity pool.
5. **Leak gate.** AddressSanitizer + LeakSanitizer enabled on all debug-build CI runs. Zero leaks tolerated.
6. **Long-session soak.** Overnight CI job: open big drawing, perform 10,000 edits with undo, save, close. RSS at the end must be within 5% of RSS at the start.
7. **Tracy profiler integration.** Every engineer's local build wires up Tracy. Frame time, allocation rate, GPU timing visible at all times. Performance work is done with measurement, not intuition.

16.5 Tools and people to bring in

- **Profilers:** Tracy (primary, every build), Intel VTune (deep CPU work), Heaptrack + Valgrind Massif (Linux memory), Visual Studio Diagnostics + WPA (Windows), AddressSanitizer + LeakSanitizer (CI gates).
- **Allocators:** mimalloc or jemalloc as a drop-in replacement for system `malloc` — 10–30% wins on allocation-heavy CAD workloads, near-zero integration cost.
- **Custom pools:** small-object pool for entity handles; arena allocator for transient command state (frees in one shot on transaction commit or abort).
- **Spatial indexes:** ODA's spatial filter for the database tier; embree BVH for the render tier if we want a ground-up alternative; both well-known, both proven.
- **Expertise sources:** Spatial CTC consultants (official ACIS memory-model guidance, bundled with the OEM contract); ex-AutoCAD / ex-Revit / ex-SolidWorks performance engineers via Toptal, Round Table Group, Guidepoint, GLG (~\$300–\$800/hr); back-channel with peer architects at Bentley / Hexagon / ANSYS SpaceClaim (other ACIS OEMs). **One day of an ex-AutoCAD perf engineer is worth a month of internal trial-and-error.** Budget for 40–80 expert-hours in Phase 1 specifically for the memory architecture.

16.6 What we deliberately don't do

- **No silent swap reliance.** If a drawing doesn't fit in budget, fail loudly. Pretending it works by paging is the worst possible customer experience.
- **No GC-based memory management for the kernel.** RAII + opaque handles. GC introduces unpredictable pauses; pauses kill responsiveness; responsiveness is the product.

- **No shared mutable state across threads without MVCC.** Every cross-thread read goes against a snapshot, never against live writable state. Locks on read paths are an outage waiting to happen at customer scale.
- **No “we’ll profile later.”** Profiling is wired in from commit 1 (Tracy + sanitizers + CI gates). Bolting performance work on after GA is what produces the “*moderately large drawings get stuck*” reputation we’re escaping.
- **No premature out-of-core / streaming database.** AutoCAD has it. Revit has it. Both are huge engineering investments — multi-person-year. We commit to **in-RAM-only** for the 3-year plan, with the budget + eviction rules that make in-RAM viable for ActCAD’s drawing-size distribution. Out-of-core is a Phase-4+ option if the customer base ever needs it; until then, the budget enforcement in Rule 7 is the right answer to “*my drawing doesn’t fit.*”
- **No giving plugins direct access to db.** Plugins go through `cmd` (§3, §13). Direct-access plugins are how every previous CAD product lost the ability to enforce these rules — once you let third-party code touch the database without a transaction boundary, every memory rule above becomes optional.

16.7 Bottom line in one paragraph

Hold the drawing in memory; that’s correct and unchanged from how every serious CAD product works.

Treat memory as a budgeted resource with a declared cap, separate the one writable layer (`db`) from the four cacheable layers above it, use MVCC so reads never block writes, build the spatial index at load time, materialize ACIS bodies lazily, evict tessellation under pressure, store undo as deltas not snapshots, put every operation over 8 ms on a worker thread, and fail loudly rather than silently swap. Wire profilers and CI gates in from commit 1 so the architecture is measured, not hoped for. Do this and a 250MB DWG with 3D solids stays responsive on a \$500 laptop — the architecture is what makes it possible, not the hardware. Skip this and you ship the same “*gets stuck on big drawings*” reputation we’re escaping, just with our name on it.

17. Next steps

1. Review and approve this plan, or push back on specific decisions in §2, §12, §13, §14, or §15.
2. Run the feasibility spike (§9) — 4–6 weeks, small senior team, dedicated.
3. Convert each locked decision in §2 into an ADR under `docs/decisions/`.
4. Stand up the engineering org for Phase 1 (engine team, shell team, agent team, LISP-shim team, infra team).
5. Lock the Phase 1 milestone definitions and the willing-customer cohort for the month-12 native Windows beta.
6. Open commercial conversations (commitments locked per §10 and §15.4 — these are execution, not decisions): **ODA Sustaining membership signup; Qt Commercial Application Development Enterprise multi-year quote (target 15–35% below first quote, with mobile / WASM add-ons and Phase-3 price protection); 2-3 Visual Studio Enterprise seats for designated Windows-perf devs; AI dev tooling subscriptions (Cursor Business / Copilot Business / Claude Code) for the whole team; Clerk + Stripe accounts.**

7. Open the ACIS commercial conversation with Spatial under NDA (Spike 1b workstream) —

request a term sheet covering: module list scoped to ActCAD use case (Local Ops + Healing + Defeaturing as floor; Polyhedral + AGM if Phase-3 BIM needs them); **royalty model — per-deployment is the primary ask, per-seat the documented fallback only** (see §15.1.1: ~10× cost variance at SMB scale, full negotiating sequence with tier breakpoints, minimum floor, and carve-outs); WASM / Linux / macOS SKU availability; source-escrow option; full DELA text; change-of-control + business-continuation clauses; CTC support-hour bundle. Legal review of the DELA before signing.

Architecture Overview & Implications

ActCAD — Architecture, Pros/Cons, and Re-Architecture Implications

Context

This repo is named `Actcad-Re-Architecture-` but currently contains only `.claude/` plugin configuration — no ActCAD source. Before any redesign work begins, we need a shared picture of:

1. What ActCAD actually *is* under the branding (the stack it's built on).
2. Where the strengths and weaknesses live in that stack.
3. Which architectural decisions the re-architecture must own.

This document is a research deliverable based on **public sources only**. The user will share internal ActCAD material (source, internal design docs, API usage telemetry, etc.) later; this write-up should be revisited and grounded against that material when it arrives.

1. What ActCAD Actually Is

ActCAD is a commercial DWG-compatible CAD product by **Jytra Technology Solutions Pvt. Ltd.** (East Godavari, India). It is **not** an independent CAD engine — it is an OEM build on top of the **IntelliCAD** engine, with Jytra applying customizations to the ITC source tree.

That single fact dictates almost everything about the architecture.

1.1 Layered Stack (bottom → top)

Layer	Component	Owner	Role
L0 Foundation	ODA Drawings SDK (formerly Teigha)	Open Design Alliance	DWG/DXF read/write, drawing database, geometric primitives, rendering primitives
L0 Foundation	ACIS 3D Kernel	Spatial (Dassault)	Boundary-representation solid modeling for 3D
L0 Foundation	Wintopo / pstoeedit	3rd-party	Raster-to-vector, PostScript-to-vector helpers
L1 Engine	IntelliCAD source (currently v14.1 in ActCAD 2027)	ITC consortium	CAD shell on top of ODA: command system, entity model, UI shell, plotting, selection, snap, OSnap, LISP runtime, etc. ITC describes the IntelliCAD source as essentially “a set of examples of how to use the Teigha/IRX interface”
L2 Product	ActCAD modifications to ITC source	Jytra	Branding, UI tweaks, vertical-app surfaces (Architecture / Mechanical / Electrical / BIM / GIS), licensing, install/update
L3 Extension API	LISP, DCL, DIESEL, SDS/ADS (C), IRX (C++), DRX, COM, VBA, .NET, plus CUI/MNU customization	ITC + ActCAD	Customer-written extensions

Jytra is a paying ITC member, so they receive full IntelliCAD source — that's how they can “make changes to match its requirements, which is not possible with any other OEM CAD engine.”

1.2 How a Drawing Flows Through the Stack

1. **Open:** ODA Drawings parses the DWG and materializes objects into the in-memory drawing database.
2. **Display:** IntelliCAD's view/render layer walks the database and pushes draw primitives through ODA's GS (graphics system) to GDI/OpenGL/DirectX.
3. **Edit:** Commands (built-in C++ via IRX, or user-written LISP/.NET) mutate the database; the GS invalidates and redraws.
4. **3D ops:** Solid modeling commands hand off to ACIS; results are stored back as ACIS B-rep blobs inside the DWG.
5. **Save:** Database serialized back to DWG via ODA, preserving R2.5 → current compatibility.

2. Pros of the Current Architecture

- **Cheap to build, cheap to sell.** Buying into IntelliCAD source + ODA + ACIS replaces ~25 years of engine R&D. Retail starts around \$199 perpetual.
- **DWG fidelity by construction.** ODA is the de facto non-Autodesk DWG implementation; format compatibility is essentially a solved problem.

- **Source-level access to the engine.** Unlike OEMs of closed engines, Jytra can patch the engine itself for ActCAD-specific behavior.
- **Mature ACIS-backed 3D.** Solid modeling, booleans, fillet/chamfer, etc. are industrial-grade and well-tested.
- **Broad extensibility surface.** LISP/DCL ports of legacy AutoCAD customizations work largely unchanged — a meaningful moat for migrating AutoCAD shops.
- **Multi-core / 64-bit native** per ITC's published platform notes.
- **Ecosystem leverage.** 25+ OEMs share the IntelliCAD platform — bug fixes and ODA upgrades flow downstream automatically.

3. Cons / Architectural Debt

3.1 Strategic (the expensive ones)

- **Cadence is not yours.** Engine major versions (IntelliCAD 14 → 14.1 → 15...) ship on ITC's timeline. Big architectural improvements ActCAD wants either wait for ITC, get carried as a vendor patch (merge pain on every uptake), or get built awkwardly above the engine.
- **Licensing stack is heavy and partly proprietary.** ACIS royalties + ODA membership + ITC fees compound per seat. Any move to undercut Autodesk on price is constrained by these floors.
- **OEM differentiation ceiling.** Most ITC OEMs ship a very similar product because they share the core; meaningful differentiation requires either deep engine forks (merge cost) or new layers above (where ActCAD's vertical apps sit today).

3.2 Technical

- **1990s C++ monolith DNA.** IntelliCAD descends from Visio/Softdesk's 1996-era "Phoenix" AutoCAD clone. The object model, command dispatch, and UI shell carry that lineage. Memory model and threading are bolted on rather than designed in.
- **API surface is huge and overlapping.** LISP + DCL + DIESEL + SDS (legacy C) + IRX (C++) + DRX + COM + VBA + .NET. Every API is a permanent maintenance contract with customers. SDS was already deprecated once (during the IntelliCAD 7 ODA transition) and still hasn't fully gone away.
- **Platform locked to Windows in practice.** ODA and ACIS are cross-platform, but the IntelliCAD shell, the .NET/COM/VBA APIs, MFC-style UI, and Jytra's product layer are Windows-centric. macOS/Linux/web are not first-class.
- **Performance ceiling on large drawings.** Public reviews repeatedly cite "moderately large drawings frequently get stuck" — symptomatic of the synchronous main-thread command/render loop typical of the IntelliCAD shell, combined with no enforced memory budget, eager kernel materialization, and linear-scan hit-testing. The re-architecture addresses this with a memory architecture committed up-front rather than retrofitted — strategic picture in [docs/rearchitecture-plan.md](#) §16; full engineering reference in [docs/memory-architecture.md](#) (five-layer model, eight commitments, MVCC concurrency, KAL contract, eviction protocol, failure modes, CI gates, anti-patterns, IntelliCAD-migration teaching).

- **Vertical apps are loosely integrated.** Architecture / Mechanical / Electrical / BIM are layered as plug-in style modules, not deeply unified — meaning data flows between them are limited.
- **UI paradigm anchored to AutoCAD ~2010.** Ribbon + command line + modal dialogs. Touch, collaboration, and cloud are retrofits.

4. Implications for a Re-Architecture

The point of writing this document before touching code is to surface the *decisions* the re-architecture must own. Each of these is a fork in the road:

4.0 Why the engine decision is forced — three lived pains

The §3 cons are the abstract case for change. What follows is the *concrete* case, drawn from real operating experience running ActCAD on top of IntelliCAD. These three pains are not separate issues — they are three faces of one root cause: **the platform layer isn't ours, so the things that need to be deeply rebuilt can't be.**

4.0.1 Rendering needs per-machine tuning and still isn't fast enough

- ActCAD's viewport renders through ODA's Graphics System (GS), historically OpenGL-based. ODA has been migrating toward Vulkan and ships a DirectX backend, but each step is on ODA's schedule, then ITC's schedule, then ours.
- Customers regularly need OpenGL driver tweaks, GPU-specific switches, and per-system tuning just to get usable performance. Even when tuned, performance ceiling is below what modern competitors deliver.
- Compare: AutoCAD 2027 shipped its own "Graphics System Framework" (GSF) — Autodesk owns the rendering pipeline end-to-end and tunes it as a single product. We cannot match that as a downstream consumer of someone else's renderer.

What this rules out: cosmetic fixes. The renderer can only be made truly fast if we own it.

4.0.2 Non-Windows platforms are not within reach

- ODA Drawings and ACIS are cross-platform at the library level. The IntelliCAD shell technically has macOS and Linux ports through ITC, but the customer-facing surface (UI shell, installer, .NET/COM/VBA APIs, LISP/IRX ABI) is Windows-shaped.
- Graebert (ARES) is the one ITC OEM that committed to Linux + macOS + web parity, and they invested at a level most consortium members will not.
- The market has moved: customers expect at minimum macOS support, and increasingly Linux and web. We cannot deliver any of those credibly on the current stack within a reasonable horizon.

What this rules out: "we'll port the existing app." The shell is too Windows-shaped to port; it has to be rebuilt with portability in the bones.

4.0.3 AI inside the codebase is structurally blocked

This is the deepest of the three, with two layers:

1. **Access.** ITC consortium license terms restrict sharing IntelliCAD source with third-party services (including AI assistants) and with outside contractors. For any non-employee engagement — code review, external development partner, AI-assisted refactor through a cloud LLM — the foundational engine is dark. We can put our patches and our product layer in front of an AI, but not the layer where the real work would be.
2. **Leverage.** Even setting access aside, we cannot land changes upstream on our own clock. Every AI-assisted refactor on the engine becomes a permanent vendor patch that we carry forever and re-merge on every ITC version uptake. The economics of AI-assisted engineering invert when you can't merge what the AI writes.

What this rules out: meaningfully accelerating engine-level work with modern tooling. AI assistance only pays off at the layers we own outright.

4.0.4 Consequence for §4.1

The §3 cons could, in theory, be argued for or against. The §4.0 pains have all been *tried* and found to not be solvable inside the current arrangement. Together they collapse the §4.1 engine decision:

- **Keep IntelliCAD** is not viable as a strategic 5-year posture. It is viable only as a near-term ship vehicle while the replacement is built.
- **Replace, build on ODA directly** is the choice the operating reality is pushing us toward — own the engine, own the renderer, own the platform abstraction, own the codebase that AI can operate on.
- **Replace with non-ODA stack** stays a non-starter (DWG fidelity loss).

The decision below in §4.1 should be read as: *replace; sequencing and migration are what's open, not the direction.*

4.1 The four big decisions

1. **Engine: keep or replace IntelliCAD?** - *Keep*: lowest risk, preserves DWG fidelity, preserves the LISP/IRX ecosystem, but inherits the monolith and the ITC cadence. - *Replace (build above ODA directly)*: ODA Drawings + ODA Visualize + own command/UI layer. Drops the monolith, but you're rewriting ~25 years of CAD shell. Realistic only with a multi-year horizon. - *Replace (non-ODA engine)*: would forfeit DWG compatibility — likely non-starter for ActCAD's audience.
2. **3D kernel: stay on ACIS, direct OEM contract. - Decided: ACIS (Spatial / Dassault), direct OEM contract.** AutoCAD's 3D solids are stored as ASM (Autodesk Shape Manager) — a fork of ACIS that Autodesk took private around 2001 — and DWGs persist them as ACIS / ASM SAT blobs. For lossless round-trip of AutoCAD-origin 3D solids — the central value prop for AutoCAD migrators — ACIS is the only kernel that doesn't introduce translation drift. - ActCAD already uses ACIS today (via IntelliCAD); the re-architecture takes the contract direct, drops the IntelliCAD middleman, and keeps customer file fidelity unchanged. - All kernel access goes through a **Kernel Abstraction Layer (KAL)** so the engine never depends on ACIS-specific types in public headers; a kernel swap stays technically possible (~6–12 weeks of focused work) if business conditions ever require it. - Alternative kernels were considered and rejected on the same basis: anything that isn't ACIS introduces translation drift on AutoCAD-origin

SAT blobs. See `docs/rearchitecture-plan.md §11.2`. - Commercial terms (initial OEM fee + maintenance + royalty + component module fees + DELA) are negotiated in Spike 1b under NDA — see `docs/rearchitecture-plan.md §15.1` for the cost structure, and §15.1.1 for the **per-deployment royalty model decision** (locked as the negotiating ask; ~10× cheaper than per-seat at ActCAD's SMB scale).

3. **API surface: which legacies survive?** - LISP almost certainly must survive — that's the migration story from AutoCAD. - SDS/ADS, DIESEL, VBA, COM are candidates for sunset. - A modern story (Python? TypeScript-in-browser? .NET 8+ cross-platform?) is the place to invest.
4. **Platform: Windows-only vs cross-platform / web.** - Web is where BIM / collaboration competition is moving (Autodesk Forma, Onshape, etc.). - Going web-native is a near-total rewrite of the UI shell — but the engine layers (ODA, ACIS) already have headless / server modes.

4.2 Risks to manage

- **DWG round-trip fidelity** — anything that touches the database layer must be measured against the existing DWG test corpus.
- **Customer extension breakage** — quantify which APIs are *actually* used in the wild before sunsetting any of them. (This is one of the first things to ask for from internal material: telemetry or estimate of LISP / IRX / .NET extension distribution among paying customers.)
- **Engine upgrade merge cost** if staying on ITC source — every ActCAD-side patch is a permanent rebase burden.
- **3D kernel migration** is irreversible in practice once shipped to customers — which is why we are *staying* on ACIS rather than migrating. The KAL preserves the technical option to swap, but the business intent is continuity.
- **ACIS commercial exposure** — opaque OEM pricing, royalty per seat or per deployment, change-of-control risk (Dassault owns SolidWorks, a competitor). Mitigated by source escrow + DELA review + business-continuation clauses in the OEM contract; see `docs/rearchitecture-plan.md §7` and §10.

4.3 What to ask of the internal material (when it arrives)

To turn this from a public-info overview into a grounded re-architecture brief, the next pass needs:

- Where ActCAD diverges from stock ITC source (file list / patch volume).
 - The build system and module boundaries inside the ActCAD codebase.
 - The vertical-app architecture (are they DLL plug-ins? statically linked? sharing a common data model?).
 - API-usage telemetry or customer-survey data on LISP / IRX / .NET / SDS.
 - Performance hotspots already known internally (large-drawing slowdowns from §3.2 — is the bottleneck the regen, the GS, the DB, or the UI?).
 - Licensing exposure: per-seat cost breakdown across ODA, ACIS, ITC.
 - Roadmap commitments already made to customers (BIM, cloud, mobile).
-

5. Verification

This deliverable is research, not code, so there is nothing to run. The check is editorial:

- §1 (stack) matches what internal docs say about ActCAD's actual layering.
- §3 (cons) is reconciled against internal performance / support data — public reviews are directional but not authoritative.
- §4 decisions are the ones the engineering leadership actually has open. If a decision in §4 is already made internally, this doc should be updated to reflect it rather than re-litigate.

Once internal material lands, re-open this file and either confirm each section or annotate where reality differs.

Sources

- [ActCAD case study — IntelliCAD.org](#)
- [ActCAD 2026 released on IntelliCAD 14](#)
- [ActCAD 2027 released on IntelliCAD 14.1](#)
- [IntelliCAD — Wikipedia](#)
- [IntelliCAD CAD Development Platform Framework](#)
- [Programming with IntelliCAD](#)
- [ActCAD product site](#)
- [Jytra Technology Solutions](#)
- [BricsCAD vs ActCAD vs AutoCAD comparison — SpotSaaS](#)
- [ActCAD reviews — SourceForge](#)

Industry Outlook

CAD Industry Outlook — ActCAD's Segment, 2–5 Year Horizon

Scope

This document looks forward 2–5 years ($\approx 2026 \rightarrow 2031$) at the slice of the CAD market ActCAD competes in:

- **Product class:** DWG-native 2D drafting plus light/mid 3D; AutoCAD-alternative positioning.
- **Pricing class:** low perpetual / affordable subscription (typically <\$500 / seat).
- **Verticals:** AEC SMBs, MEP, Electrical, Mechanical drafting, GIS, Civil/Survey.
- **Geographies:** India + SEA + LATAM + EU SMBs + NA cost-conscious.
- **Direct peers:** BricsCAD, ZWCAD, GstarCAD, progeCAD, ARES (Graebert), DraftSight, NanoCAD; plus free/open (LibreCAD, FreeCAD, NanoCAD Free).

It is paired with `docs/architecture-overview.md`. Every trend below ends with an implication that feeds §4 of that document.

1. Five forces reshaping the segment

1.1 AI assistance becomes table stakes (12–24 months)

- AutoCAD 2027 ships native integration with Anthropic's Model Context Protocol so the editor exposes drawing context to LLM agents.
- Graebert ARES 2027 ships **A3** (AI assistant) with multi-task prompting, AI-generated blocks, command recommendations, and voice interaction in Kudo.
- Industry estimates put AI handling 20–30 % of routine drafting work today, growing fast.
- By 2028, products without an in-editor AI assistant will read as legacy to evaluators.

Implication for ActCAD re-arch: AI is no longer a feature; it's a layer. The new architecture must expose a stable, structured representation of the drawing database to an LLM/agent runtime — an MCP-style server or equivalent — *before* anything else gets modernized. This is the single most consequential decision in §4.1 of the architecture doc, ahead of the “keep or replace the engine” question.

1.2 Cloud + web + mobile parity becomes baseline (24–36 months)

- AutoCAD 2027 introduced real-time multi-user editing of the same DWG plus Forma cloud integration.
- ARES delivers near-feature-parity desktop + web + mobile + Linux — the line industry analysts now use to define “modern” CAD.
- Onshape and Fusion's web app already define “cloud-native CAD” (app + database in cloud, microservices, multi-tenant co-edit) as the reference posture for new entrants.

- Quote that frames the period: “In 2026, cloud-based CAD collaboration is no longer a nice-to-have — it is the standard.”

Implication for ActCAD re-arch: even if the primary deliverable stays a desktop binary, the *database and command model* must permit a server-side / multi-user mode. Decisions in §4.1 (“engine: keep or replace”) and §4.4 (“Windows-only vs cross-platform / web”) of the architecture doc become joined — the engine choice has to leave a path to web. The Graebert ARES + ARES Kudo pattern (shared engine, three UIs) is the most credible reference architecture for ActCAD’s size.

1.3 BIM / CAD convergence moves downmarket (24–60 months)

- Global BIM market growing ~14.7 % CAGR through 2028.
- Mandated BIM adoption rising globally; estimates put adoption at ~80 % by 2026 and ~90 % by 2028 in mature regions.
- SMB BIM adoption is blocked by license + hardware cost (often >\$50 k / firm) and by the trained-staff shortage (48 % of construction firms cite hiring difficulty).
- That gap — “BIM-light for SMBs” — is the largest underserved opportunity in this segment.

Implication for ActCAD re-arch: invest in ActCAD’s BIM vertical at a multiple of investment in pure 2D drafting. The architecture must let BIM share a unified object model with 2D drawing entities (not a bolted-on side database), so SMB users can climb the BIM curve incrementally rather than buying a separate product. Revisit §3.2 of the architecture doc — “Vertical apps are loosely integrated” — this is the bullet that hurts most in the 5-year horizon.

1.4 Pricing models fragment, perpetual partially returns (ongoing)

- Subscriptions hit 68 % of new US CAD purchases in 2024.
- Counter-trend in 2026: notable customer-side migration *back* to perpetual where available, driven by subscription fatigue and Autodesk’s named-user enforcement.
- Indian SME market, conversely, is moving *toward* subscription — low IT budgets favour opex.
- Net direction is hybrid: perpetual base license + optional cloud/AI subscription add-on.

Implication for ActCAD re-arch: ActCAD’s perpetual model is an asset, not a liability — but only if the architecture cleanly separates the *local desktop core* (perpetual) from *cloud-delivered services* (AI assistant, co-edit, asset library, twin-data sync). The license boundary needs to be an architectural seam, not a business afterthought. This becomes a fifth bullet in §4.1’s decisions.

1.5 Digital twin + IoT enters the SMB MEP conversation (36–60 months)

- MEP digital twins via IoT — sensors livestreaming into the BIM model — are normalizing for facilities operators.
- AI-driven segmentation of point clouds / reality-capture scans is collapsing the time to produce as-built MEP models.
- Currently big-firm / big-budget, but the tooling stack (ODA’s ifc/point-cloud SDKs, AI segmentation libraries, MQTT/IoT brokers) commoditizes fast.

Implication for ActCAD re-arch: the MEP and Electrical verticals — the most loyal portion of ActCAD’s user base — need a 3-year story for ingesting reality-capture data and a 5-year story for streaming IoT data into the model. The object model needs an extensible “non-CAD payload” hook from the start, not retrofitted.

2. Where ActCAD’s user base is going

User segment	2–5 year direction	ActCAD posture
AEC SMBs (architects)	Pulled toward BIM, priced out of Revit; want Revit-lite	Invest — best fit for BIM-light bet (\$1.3)
Mechanical drafting / job-shop manufacturing	Pulled toward Onshape / Fusion cloud-MCAD; desktop DWG share declines	Defend, don’t grow — table-stakes MCAD, not a growth area
Electrical / MEP contractors	Most loyal to ActCAD-class tools; digital-twin overhang	Invest — verticalize hard; this is the moat
Civil / survey	Increasingly tied to Carlson, Civil 3D, OpenRoads	Partner or exit — not a defensible standalone position
GIS	Losing share to QGIS, ArcGIS Pro	De-emphasize — not where to put R&D dollars

3. Geographic forecast

- **APAC** ≈ 40 % of new CAD-startup funding globally; the gravitational center of the SMB CAD market is moving east.
- **India** SMB market is shifting to subscription / cloud — ActCAD’s home turf but actively contested by ZWCAD (China) and GstarCAD; home-field advantage is real but not durable without product differentiation.
- **EU** consolidating around Graebert (ARES) for mid-market DWG-CAD; perpetual licensing remains viable.
- **NA** subscription fatigue creates a window for low-cost perpetual, but evaluators now expect AI + cloud as baseline.

4. What this means for the re-architecture

Mapping each trend back to the decisions in [docs/architecture-overview.md](#) §4:

Trend	Affects decision	New posture
\$1.1 AI assistance	\$4.1 engine choice; new “AI/agent surface” decision	AI/agent API is the first modernization layer, ahead of engine swap
\$1.2 Cloud parity	\$4.1 engine + \$4.4 platform	Engine choice must preserve a server-side / multi-user path
\$1.3 BIM downmarket	\$3.2 vertical integration	Unified object model across 2D + BIM; vertical apps move from plug-ins to first-class
\$1.4 Hybrid pricing	new decision: license seam	Architectural separation of local-perpetual core vs cloud-subscription services
\$1.5 Digital twin / IoT	\$4.2 3D kernel and object model	Object model must allow non-CAD payloads (sensor streams, reality-capture meta) without engine forks

The bigger picture: a 2-year-out ActCAD that is just “cheaper AutoCAD with AI sprinkled in” is the losing position, because every peer (BricsCAD, ZWCAD, ARES, GstarCAD) will be there too and Autodesk will outspend everyone on AI. A defensible 5-year-out position is **“AI-assisted, cloud-collaborative, BIM-capable CAD for SMB AEC and MEP, priced perpetual + optional cloud subscription, India-first”**. The architecture work is what makes that position physically possible.

Sources

- [Autodesk AutoCAD 2027 — Architosh](#)
- [ARES 2027 deep dive — Architosh](#)
- [Graebert ARES 2027 + Forma — Architosh](#)
- [7 CAD design trends 2026 — Shalin Designs](#)
- [CAD drafting trends 2026 — CADdrafter](#)
- [AI in CAD 2026 — Fabrixon](#)
- [5 BIM trends 2026 — United-BIM](#)
- [BIM adoption worldwide 2026 — Novatr](#)
- [BIM software market 2034 — Dataintel](#)
- [Cloud-native CAD features — Onshape](#)
- [Cloud CAD collaboration 2026 — Orbit Training](#)
- [India CAD software market — PS Market Research](#)
- [AutoCAD perpetual license 2026 — ZWSOFT](#)
- [Perpetual vs subscription 2026 — DEV.to](#)
- [Digital twin for MEP — Autodesk University](#)
- [Electrical digital twin market — BRI](#)

Memory Architecture — Engineering Reference

ActCAD Memory Architecture

Status: Architecture document. Companion to `docs/rearchitecture-plan.md §16` (executive summary). This document is the engineering reference — read it before writing code that touches `db`, `geom`, `render`, or any cache.

Table of contents

1. Why this document exists
2. Goals and non-goals
3. The CAD memory model in plain terms
4. The five-layer model
5. The eight commitments — full detail
6. Concurrency model in depth
7. The memory-budget machinery
8. The Kernel Abstraction Layer (KAL)
9. Integration with other modules
10. WASM and the browser shell
11. Mobile implications (Phase 3)
12. Failure modes and recovery
13. Telemetry and observability
14. Testing strategy
15. CI gates — exact specifications
16. Tools
17. Expertise sourcing
18. Anti-patterns — what we will not allow
19. Migration from IntelliCAD patterns
20. Phase-by-phase rollout
21. Open questions
22. Glossary
23. References

1. Why this document exists

The single most consistent complaint in public reviews of ActCAD today is some variation of: “*moderately large drawings frequently get stuck*”, “*freezes on opening big files*”, “*hangs when saving*”, “*crashes after a long session*”. That reputation is inherited from the IntelliCAD shell — synchronous main-thread command + render loop, no enforced memory budget, eager kernel materialization, linear-scan hit-testing in places where indexes should exist.

The re-architecture exists in significant part to escape that reputation. Memory architecture is the place where this is either won or lost. Geometry kernels are fast; renderers are fast; the bottleneck is almost always how the application manages the working set of an active edit session.

This document is the engineering commitment for how the new ActCAD engine manages that working set. It is not a roadmap, not a feature list, and not a survey of options. **It is the rules.** Every engineer joining the team should read this once and know what is allowed, what is forbidden, and which symptoms in the code or in customer reports map to which architectural commitment.

The executive summary lives in `docs/rearchitecture-plan.md` §16. Strategic and budget framing lives in `docs/exec-presentation.md`. This document is the depth.

2. Goals and non-goals

2.1 Goals

1. **Predictable responsiveness across drawing-size distribution.** A 250 MB DWG with 500 3D solids opens, edits, and saves at the same UI fluidity as a 5 MB DWG with 50 entities. Performance degrades gracefully, not in a cliff.
2. **Bounded memory footprint with declared, enforced limits.** Working set is a budget, not an emergent property. Exceeding the budget triggers loud, recoverable failure, never silent paging.
3. **Concurrent edit + render + agent query, with no readers blocked by writers.** The UI stays responsive during a long ACIS boolean. The AI agent can query state during an edit. Co-edit peers replicate without lock storms.
4. **Crash-resilient long sessions.** 8-hour edit sessions don’t accumulate heap. Undo storage is bounded. Tessellation cache cannot starve the database.
5. **Measured, not hoped for.** Every commitment in §5 is enforced by CI gates from commit 1, not retrofitted after GA.

2.2 Non-goals

1. **Out-of-core / streaming database.** AutoCAD has one. Revit has one. They are multi-person-year investments. We commit to in-RAM-only for the 3-year plan with the budget enforcement that makes in-RAM viable for ActCAD’s drawing-size distribution. Out-of-core is a Phase-4+ option only.
2. **Lock-free everything.** We use locks where they’re the right tool. The rule is *no reader blocks on a writer*, not *no thread ever waits*.
3. **GPU-resident db.** Tessellation lives on the GPU. The drawing database does not. ResizeBAR / GPU-direct-storage are not in scope.

4. **Custom OS-level memory manager.** We use the OS allocator (or mimalloc as a drop-in) plus targeted object pools and arenas. We do not write a slab allocator from scratch.
5. **Pluggable kernels in v1.** The KAL keeps a kernel swap technically possible at ~6–12 weeks of focused work. It is not a runtime-swappable abstraction. ACIS is the only kernel that ships.

2.3 Success criteria

A 250 MB DWG with 500 3D solids, opened on a 16 GB / 8-core developer laptop:

- **Open time** ≤ 8 seconds, of which $\geq 80\%$ is disk I/O + DWG parse, $\leq 20\%$ is index build, 0% is eager kernel materialization
- **Steady-state RSS** $\leq 3\times$ the on-disk DWG size (target $\leq 2\times$ for AEC drawings without dense 3D)
- **Pan / zoom / hover P99 frame time** ≤ 16 ms during normal interaction; ≤ 33 ms during background regen
- **Save time** ≤ 4 seconds with a progress bar; UI remains interactive
- **Long-session test** (10,000 edits + undos across 8 hours) ends with RSS within 5% of the post-load baseline
- **Out-of-budget drawing** produces a clear modal: *“This drawing requires X GB; your current budget is Y GB. Close other drawings or raise the limit in Preferences.”* — never a force-quit-and-lose-work situation

3. The CAD memory model in plain terms

3.1 Open-once, edit-in-memory, save-back

Every serious CAD product since AutoCAD R1 (1982) follows the same shape: open the drawing once, hold it entirely in memory, do all reads and writes against that in-memory state, write back to disk on save. This is not negotiable and not up for redesign. Out-of-core databases exist (Bentley MicroStation has one, Revit has one) but each represents many engineer-years of investment we do not take on.

What this model gives us:

- **O(1) random access** to any entity by handle
- **Transactional semantics** — a single user gesture maps to a single durable mutation
- **Single source of truth** for renderers, agents, plugins, and the network layer
- **No I/O on the read path** — every query is in-RAM

What this model demands of us:

- The entire working set fits in process memory, or we refuse to open
- Every cache or derived structure must declare its budget and respect a global cap
- Every consumer that holds a reference to a long-lived entity participates in the lifecycle protocol

3.2 Why naive in-memory blows up

The naive implementation of “open and hold in memory” loads a 250 MB DWG and assumes the process will use ~250 MB. In practice:

- The DWG **parses to ~2–3×** its on-disk size due to handle tables, attribute dictionaries, xref expansion
- Every 3D solid touched by the user **materializes a kernel ENTITY graph** that can be 5–50× the SAT blob size
- Every rendered entity has **GPU-resident tessellation** that scales with view detail
- Every edit appends to an **undo log** that, if implemented as snapshots, can grow unbounded over a long session
- The **renderer caches** invalidation regions, frustum-cull results, and material binds independently
- **Spatial indexes** add 5–10% on top of the entity model
- **Plugins** allocate against the same heap with no accounting

Result: a 250 MB DWG can occupy 4–6 GB of working set, then page when the OS notices, then appear “frozen” to the user, then get force-quit. **Every one of these failure modes is preventable. None is prevented by the IntelliCAD shell pattern we are leaving behind.**

3.3 What “the drawing in memory” actually contains

The phrase “the drawing in memory” is misleading. There is no single object. There are five distinct things, each with its own size, owner, lifecycle, and pressure-response policy. The architecture in §4 makes this separation explicit so that under pressure, the eviction story is mechanical rather than emergent.

4. The five-layer model

Layer	Name	What lives here	Typical size on a 250 MB DWG with 3D solids	Owner module	Pressure response
1	DWG database	Entities, handles, layers, blocks, xrefs, dimstyles, attribute dictionaries, the on-disk SAT blob for every 3D solid (before kernel materialization)	400–800 MB	db	Never evicted — single source of truth
2	ACIS kernel bodies	The materialized ENTITY graph for every 3D solid the user has touched since open	200 MB – 2 GB	geom via KAL	Partial — body graphs are lazy-loaded on first geometric operation; non-essential attributes can be discarded
3	Tessellation cache	Triangle meshes + GPU vertex / index buffers for everything currently or recently in view	500 MB – 4 GB	render	Yes — LRU evict on pressure; regenerate from B-rep on next view
4	Spatial indexes	R-tree / BVH for hit-test, snap, frustum cull, selection-window, agent spatial queries	50–200 MB	db (entity index) + render (view index)	Rebuildable — can be discarded and recomputed from Layer 1
5	Undo / op-log	Delta records: every committed transaction's inverse, plus replication and audit fan-out	100–500 MB over a long session	db	Truncatable — drop oldest beyond a configurable threshold; warn before truncation

4.1 Layer 1 — DWG database (db)

The canonical entity model. Held in process memory for the lifetime of the open document. Mutations go through one transaction queue; reads go through MVCC snapshots (§6). Layer 1 is the **only writable layer** — every other layer is a cache or derived view.

Layer 1 includes the on-disk SAT blob for each 3D solid, **kept in its parsed-but-not-materialized form** until first geometric operation. This is the lazy boundary between Layer 1 and Layer 2.

Eviction policy: none. Layer 1 is never evicted. If pressure forces a choice between Layer 1 and any other layer, every other layer evicts first.

Size discipline: Layer 1 is bounded by the DWG file content itself. Growth during a session is bounded by user edits (which append to Layer 5 as deltas, not by inflating Layer 1).

4.2 Layer 2 — ACIS kernel bodies (`geom` via KAL)

The materialized `ENTITY` graph for each 3D solid the user has performed a geometric operation on. ACIS supports lazy attribute loading natively; we plumb this through the KAL (§8.3) so the engine never materializes 500 solids on file open just because the file contains 500.

Eviction policy: partial. When pressure hits, the KAL can drop derived attributes (tessellation hints, healing metadata) but **not the body graph itself** if any consumer holds a reference. Bodies with no active references can be fully unloaded back to SAT-blob form in Layer 1.

Reference counting is enforced at the KAL boundary. The KAL holds the only `ENTITY*` pointers; the rest of the engine only sees opaque handles. This is what makes pressure-driven unload safe.

4.3 Layer 3 — Tessellation cache (`render`)

Triangle meshes + GPU vertex / index buffers for everything in or near view. Backed by the render backend's native GPU memory (DirectX / Vulkan / Metal / WebGPU).

Eviction policy: LRU with a hard cap. Default cap: 30% of process working set, or 1.5 GB, whichever is smaller. The cap is enforced *before* any allocation that would exceed it — evict-then-allocate, never allocate-then-evict.

Regeneration: any evicted tessellation can be regenerated from the cached B-rep in Layer 2 (or from Layer 1's SAT blob if Layer 2 was also evicted). Regeneration is a worker-thread task with a progress signal to the render frame loop.

4.4 Layer 4 — Spatial indexes (`db` + `render`)

Two kinds:

- **Entity spatial index** in `db` — R-tree over entity bounding boxes; serves hit-test, snap, selection-window, agent “find entities in region” queries. Built at file load (§5.3).
- **View spatial index** in `render` — BVH over visible tessellation; serves frustum cull and ray-cast for picking under the cursor. Rebuilt on view change.

Eviction policy: rebuildable. Either index can be discarded and recomputed. The entity index rebuild cost is proportional to entity count (single-digit milliseconds for 100K entities); the view index rebuilds incrementally as the camera moves.

4.5 Layer 5 — Undo / op-log (`db`)

A delta record for every committed transaction. The same op-stream that drives:

- **Local undo / redo** — apply the inverse delta
- **Co-edit replication** — ship the delta to the server for fan-out (`docs/rearchitecture-plan.md` §12)
- **AI agent context** — the agent observes the op-stream to track what changed since its last query
- **Audit** — security-relevant operations log to the audit sink

One stream, five consumers. Building it any other way means writing the same plumbing five times.

Eviction policy: truncate-oldest. Configurable threshold (default: 10,000 operations or 200 MB, whichever first). When approaching the threshold, the user is warned that further undo will lose history. Replication and audit consumers checkpoint independently and are not affected by local truncation.

4.6 Ownership matrix

Concern	db	geom (KAL)	render	cmd	agent	net	script / plugin
Mutate entity state	✓ (only)	✗	✗	✗ (mediates)	✗	✗	✗
Read entity state	✓ (live)	✓ (snapshot)	✓ (snapshot)	✓ (snapshot)	✓ (snapshot)	✓ (snapshot)	✓ (snapshot via cmd)
Hold kernel ENTITY*	✗	✓ (only)	✗	✗	✗	✗	✗
Own tessellation cache	✗	✗	✓ (only)	✗	✗	✗	✗
Own undo log	✓ (only)	✗	✗	✗ (appends via txn)	✗	✗	✗
Open spatial-index queries	✓	✗	✓ (view index)	✓	✓	✗	✓ (via cmd)
Trigger eviction	✗	✓ (kernel bodies)	✓ (tess)	✗	✗	✗	✗

4.7 Eviction order under pressure

Fixed. Not configurable per-customer because mistakes here cost data integrity.

Pressure detected

- ↓
1. Evict LRU tessellation (Layer 3) until below 70% of layer cap
 - ↓ still under pressure?
2. Drop view spatial index (Layer 4 render) – will rebuild on next camera change
 - ↓ still under pressure?
3. Unload ACIS bodies with no active references (Layer 2 → Layer 1)
 - ↓ still under pressure?
4. Truncate oldest entries from undo log (Layer 5), warning the user
 - ↓ still under pressure?
5. Drop entity spatial index (Layer 4 db) – will rebuild on next query
 - ↓ still under pressure?
6. ****Fail loudly to the user**** (\$7.5) – Layer 1 is never evicted

Layer 1 stays put. The user's work is never at risk from a memory-management decision.

5. The eight commitments — full detail

5.1 Single mutator: `db` owns all drawing state

Rule. `db` is the only module that holds writable drawing state. Every other module's view of the document is either (a) read-only against a snapshot, or (b) a derived cache that can be discarded.

Why. When something is wrong in memory, you always know who to ask. When you want to invalidate everything and start fresh, you blow away every cache and Layer 1 is intact. When two modules need to agree on what the document says, they look at the same source.

Enforcement. Public mutation API on `db` is a small surface (`begin_transaction` , `add_entity` , `remove_entity` , `modify_entity` , `commit_transaction` , `abort_transaction`). No other module exports a function that returns a writable handle to a `db` entity. Code review rejects any PR that introduces one. CI rule (clang-tidy custom check) flags non-`db` modules that return non-`const` references to entity types.

Tradeoff accepted. Every cross-module change to the document costs a transaction round-trip. This is the price of integrity and we pay it cheerfully.

5.2 MVCC: reads never block writes (and writes never block reads)

Rule. Mutations on `db` happen on one worker thread, serialized through a transaction queue. Reads happen on any thread, against a **copy-on-write snapshot** of the committed state.

Why. This is the single most important commitment in the architecture. It is what makes the UI stay at 60fps while a long ACIS boolean runs, what makes the AI agent able to query state during an edit, what makes co-edit possible without lock storms. The pattern is decades-old (PostgreSQL, every modern RDBMS); the CAD industry has been slow to adopt it because CAD databases were historically single-threaded.

Mechanics (§6 has the full detail). Every entity is internally versioned. A snapshot is a (lightweight) reference to a version vector. Mutations write new versions; old versions are reclaimed once no snapshot references them. Snapshots are cheap to acquire ($O(1)$) and cheap to release ($O(1)$ amortized).

What this is not. It is not optimistic concurrency control — there are no merge conflicts in this layer because mutations are serialized. It is not lock-free — there are locks, but they're short and on the write path. It is not generic CRDTs — see §6.4.

Enforcement. Every public read API on `db` takes a `Snapshot` parameter. No public read API operates on “current state.” Code that wants “current” calls `db.snapshot()` to acquire one. CI rule rejects any read API that omits the snapshot parameter.

5.3 Spatial index built at load time

Rule. The moment a drawing finishes parsing, the entity R-tree is built. Hit-test, snap, frustum cull, selection-window, and agent spatial queries all go through the index. **Linear scan over entities is not a public API on `db`.**

Why. This is the single decision that separates snappy from unusable on a 250K-entity drawing. The index build is a one-time cost at load (single-digit milliseconds per 10K entities); without it, every hover, every pick, every snap is $O(N)$.

Mechanics. R-tree built bottom-up from entity AABBs. Updates on commit: each transaction's modified-entity list drives $O(k \log N)$ index updates where k is the modified count.

Enforcement. `db` does not export `for_each_entity()`. Spatial queries are `db.query(region, filter)`. Code review rejects any traversal that bypasses the index for performance-sensitive paths. CI gate measures hit-test P99 on the synthetic drawing corpus.

5.4 Lazy materialization of ACIS bodies

Rule. ACIS bodies stay as opaque SAT blobs in `db` until the first geometric operation on that body. Open time is bounded by parse + index, **not** by kernel materialization.

Why. A drawing with 500 3D solids that the user never touches should not pay 500× the kernel-materialization cost on open. The user clicks one solid; we materialize one solid.

Mechanics. The KAL `Body::ensure_materialized()` is called by every geometric operation. If the body is already a kernel `ENTITY`, no-op. If it's still a SAT blob, parse + materialize on the calling thread (typically a worker), record the materialization in the KAL's reference map.

Eviction. Materialized bodies with no active references can be unloaded back to SAT-blob form under pressure (Layer 2 → Layer 1). The KAL tracks references via opaque handle counts.

Enforcement. KAL is the only module that calls into ACIS. No `ENTITY*` appears in any public header outside the KAL. CI rule (header grep) fails if `ENTITY` or any ACIS type appears in a non-`geom` header.

5.5 GPU-resident, evictable tessellation

Rule. Once a body is tessellated, the triangles go to GPU memory (vertex + index buffers in the active render backend) and stay there for the render loop. Under CPU memory pressure, LRU tessellation is evicted; on next view, it's regenerated from the cached B-rep.

Why. GPU-resident tessellation is what makes the render loop run at GPU speed instead of bus speed. Eviction policy keeps Layer 3 bounded so it can never starve Layer 1.

Mechanics. Render backend abstraction (`docs/rearchitecture-plan.md` §4) presents a uniform vertex buffer API across DirectX 11/12, Vulkan, Metal, WebGPU. Tessellation cache is keyed on (body handle, view-detail level). LRU tracks last-frame-used.

Hard cap. Default 30% of process working set, or 1.5 GB, whichever is smaller. Tunable per-platform (mobile defaults are lower; see §11). Allocation that would exceed the cap evicts first.

Regeneration. Worker-thread task. The render loop draws a placeholder (wireframe, low-LOD proxy) while regeneration is in flight. Frame time stays under 16 ms even during a regeneration burst.

5.6 Delta-based undo

Rule. Every undoable operation stores its inverse, not a snapshot of the world. Reversing `move(handle, dx, dy)` is `move(handle, -dx, -dy)` — bytes, not megabytes.

Why. Full-state snapshots are what blow heap in long sessions. AutoCAD survives 8-hour edit sessions because undo is incremental. We do the same.

The op-stream is the universal substrate. The same delta record drives:

1. **Local undo / redo**
2. **Co-edit replication** (docs/rearchitecture-plan.md §12)
3. **AI agent context** (docs/rearchitecture-plan.md §14)
4. **Audit log** (security-relevant ops only)
5. **Render invalidation** (which regions need redraw)

Mechanics. Each delta is (op_id, timestamp, txn_handle, inverse_payload, forward_payload) . The forward payload is what was applied (for replication); the inverse is what undo applies. Both are typed FlatBuffers messages.

Truncation. When the op-log exceeds threshold, oldest entries are dropped. User sees a warning before truncation. Replication and audit consumers checkpoint independently — local truncation doesn't lose their state.

5.7 Declared memory budget; fail loudly, never silently swap

Rule. The engine boots with a declared memory budget. Each cache layer has a sub-budget. When a sub-budget is exceeded, the layer evicts per §4.7. When the total budget is exceeded after eviction, **the engine fails loudly to the user** with a clear modal — never silently swaps to disk.

Why. Silent swap is the worst possible failure mode. The user thinks the app is frozen, force-quits, and loses work. Loud failure is recoverable; silent swap is a support ticket and a churn risk. “*The app is slow*” is a 1-star review. “*The app told me I needed more RAM and pointed me to the setting*” is a question we can answer.

Budget formula (default).

```
budget = min(
    2 × loaded_DWG_size_on_disk,
    process_working_set_limit - OS_headroom
)
```

Where OS_headroom = 1 GB on desktop, 512 MB on mobile, 256 MB in WASM. User can raise the cap in Preferences up to the OS limit; reducing below the working-set requirement triggers a refusal to open.

Sub-budgets. Layer 1 has no sub-budget (it's whatever the document needs). Layer 2 capped at 50% of remaining budget. Layer 3 capped at 30%. Layers 4 + 5 each capped at 10%.

Loud-failure UX. Modal dialog:

“This drawing requires X.X GB of memory. Your current budget is Y.Y GB. Options: - Close other open drawings (currently using Z.Z GB) - Raise the memory budget in Preferences → Performance - Open this drawing in a separate ActCAD process (uses fresh budget) - Open in read-only mode (uses 60% less memory)”

No silent failures. No vague “out of memory” errors. Always a recoverable action.

5.8 8 ms UI-thread budget

Rule. The UI thread does input + render only. Every operation that could take more than 8 ms goes to a worker. Frame budget asserts in debug builds: any UI-thread work over 8 ms triggers a debug break.

Why. “Slow on the UI thread” is a build-breaking error from commit 1. Customers don’t file tickets for “the cursor lagged for 30 ms” — they just churn. Profilers in CI catch this; debug-build asserts catch it locally before commit.

What goes to a worker.

- File I/O of any kind (open, save, plot)
- ACIS operations (boolean, fillet, sweep, blend)
- Regen (full or partial)
- Index build / rebuild
- Tessellation generation
- Network round-trips (sync, agent, telemetry)
- Plugin operations longer than 1 ms (enforced at the `cmd` boundary)

What stays on the UI thread.

- Mouse / keyboard input parsing
- Snapshot acquisition ($O(1)$)
- Spatial index queries (sub-millisecond on a 100K-entity drawing)
- Render command submission (GPU-resident; CPU work is bounded)
- Cursor / selection-marquee updates
- Property panel redraws against the current snapshot

Frame budget assertion. Debug builds wrap each main-loop iteration with a timer; if the iteration exceeds 8 ms, the assertion logs the call stack and breaks into the debugger. Release builds log to telemetry without breaking. CI gates on P99 frame time prevent regression.

6. Concurrency model in depth

6.1 Threads we use

Thread	Count	Responsibility
UI	1	Input parsing, render-command submission, snapshot acquisition, fast read-only queries
Mutation worker	1	All db mutations, serialized through the transaction queue
Render worker	1	Tessellation regeneration, GPU buffer uploads, view-index incremental updates
Kernel worker pool	2–8 (CPU-count dependent)	ACIS operations (booleans, fillets), heavy geometric computation
I/O worker	1	File open, save, plot, telemetry flush
Net worker	1	Sync, agent, MCP, document service
Agent worker	1	MCP request handling, tool dispatch

Total at idle: 8–14 threads, of which most are blocked on a condition variable. Active threads at any moment in a typical edit session: 2–4.

6.2 The transaction queue

The mutation worker pulls one transaction at a time from a single-producer-multi-consumer queue. Every command (from `cmd`) that mutates state enqueues a transaction; the worker dequeues, executes, validates, and commits.

```

cmd (UI thread or worker)
  ↓ enqueue
transaction queue
  ↓ dequeue (serialized)
mutation worker
  1. begin_transaction() → working_copy
  2. apply ops           → working_copy
  3. validate            (referential integrity, kernel checks)
  4. commit_transaction() → publish new snapshot version
  5. emit op-stream record → consumers (undo, sync, agent, audit, render)

```

Why a single mutation thread instead of fine-grained locking.

- Serialization eliminates an entire class of race conditions
- The mutation worker is throughput-bound, not latency-bound — adding more threads doesn't help when the bottleneck is $O(\log N)$ index updates
- Validation logic is simpler when mutations are serial
- ACIS itself is largely single-threaded within one body — multi-threading the mutation worker would not help kernel operations

6.3 Snapshot mechanics (copy-on-write)

Every entity carries an internal version counter. When a transaction commits, modified entities get incremented versions; the previous versions stick around in a per-entity version chain.

A `Snapshot` is a (lightweight) timestamp plus a reference count. Acquiring a snapshot is `O(1)`. Reads through a snapshot walk the version chain to find the largest version $\leq$ snapshot timestamp.

Old versions are reclaimed when no live snapshot references them (reference-counted GC at the snapshot level, not the entity level). The reclamation happens on the mutation worker after commit; it does not block readers.

Snapshot lifetimes.

- **Frame snapshot** — render thread acquires at the start of each frame, releases at frame end. Lifetime: ~16 ms.
- **Query snapshot** — hit-test, snap, agent query. Lifetime: < 1 ms typically.
- **Long-running snapshot** — agent that wants a stable view during a multi-step plan; co-edit peer that's applying a remote op-stream. Lifetime: seconds to minutes. Bounded by a maximum snapshot age (default: 30 seconds) after which the snapshot is forcibly released and re-acquired.

Memory cost. Version chains are bounded by the snapshot horizon. Typical overhead: 1.05–1.15× the base entity size. Under sustained edit load with no long-running snapshots, overhead converges to ~1.02×.

6.4 The op-stream as the universal substrate

The op-stream emitted by `db` on every commit serves five consumers:

1. **Undo** — stores the inverse delta in Layer 5
2. **Co-edit replication** — `net` ships the forward delta to the server (`docs/rearchitecture-plan.md` §12)
3. **AI agent context** — `agent` observes the stream to track changes between queries (`docs/rearchitecture-plan.md` §14)
4. **Render invalidation** — `render` reads the stream to know which spatial regions need redraw
5. **Audit** — security-sensitive operations (signed save, license validation) log to the audit sink

Stream guarantees.

- **Ordered** — strictly in commit order
- **Lossless to subscribers** — each consumer has a checkpoint; if a consumer falls behind, the stream backpressures (rare; mutation throughput is bounded)
- **Typed** — FlatBuffers schema with versioning
- **Replayable** — given a checkpoint, the stream can be replayed to rebuild downstream state

What this is NOT.

- Not a generic CRDT — see `docs/rearchitecture-plan.md` §2.7 for the co-edit model
- Not eventual-consistency — the stream is server-authoritative at the network boundary
- Not append-only forever — Layer 5 truncates oldest entries past threshold

7. The memory-budget machinery

7.1 Budget declaration and discovery

On engine boot, the runtime queries:

- **Process working set limit** — OS-specific (Windows: `GetProcessMemoryInfo` ; macOS / Linux: `getrlimit(RLIMIT_AS)` ; WASM: `wasm_memory_max`)
- **Available physical RAM** — for the loud-failure UX hint
- **User Preferences override** — defaults to “*computed*”, can be raised to a hard cap

Budget formula (default):

```
hard_cap = min(
    user_preference_cap (if set),
    OS_working_set_limit - OS_headroom
)

soft_target = min(
    2 × sum(open_DWG_sizes_on_disk),
    hard_cap
)
```

`soft_target` drives eviction. `hard_cap` drives loud failure.

7.2 Sub-budgets per layer

Defaults, expressed as fractions of `(hard_cap - Layer_1_actual)` :

Layer	Default sub-budget
Layer 2 (ACIS bodies)	50%
Layer 3 (tessellation)	30%
Layer 4 (indexes)	10%
Layer 5 (undo log)	10%

Sub-budgets are advisory in the sense that any layer can briefly exceed its share if others are below theirs. They are enforcing in the sense that `total ≤ hard_cap` is hard.

7.3 Eviction protocol

When a layer is asked to allocate and would exceed its sub-budget:

1. Layer attempts internal eviction (LRU or layer-specific policy)
2. If still over: layer requests budget extension from the engine's memory broker
3. Memory broker checks global state; if other layers are under their sub-budget, extension granted
4. If no slack available globally: broker triggers pressure-cascade per §4.7

The broker is a single small module (`mem_broker`) owned by `platform` . It is the single point of truth for budget state. All allocations go through it for accounting (it doesn't replace `malloc` ; it tracks).

7.4 Pressure signals and OS APIs

The engine subscribes to OS memory pressure notifications:

- **Windows:** `CreateMemoryResourceNotification` + `WM_COMPACTING`
- **macOS:** `dispatch_source_create(DISPATCH_SOURCE_TYPE_MEMORYPRESSURE)`
- **Linux:** `cgroup memory.pressure_level` (where available); fallback to RSS polling
- **iOS / Android:** OS lifecycle callbacks (`didReceiveMemoryWarning` , `onTrimMemory`)
- **WASM:** allocator failure (no OS signal; we react to `malloc` returning null)

On any pressure signal, the broker triggers a level-appropriate eviction cascade before the OS starts paging.

7.5 Loud-failure UX

When the engine cannot satisfy a memory request after exhausting eviction:

1. The operation that triggered the allocation is **aborted cleanly** (transactional — no partial state)
2. The UI surfaces a modal with **concrete options** (close other drawings, raise budget, separate process, read-only mode)
3. The current document state is **untouched** — Layer 1 is intact, all committed work is preserved
4. Auto-save fires (if enabled) before the modal appears, as a belt-and-braces measure

What we never do:

- Display “*Out of memory*” with no actionable guidance
- Silently degrade to disk-backed mode without telling the user
- Force-quit
- Lose any committed mutation

8. The Kernel Abstraction Layer (KAL)

8.1 What the KAL hides

The KAL is the only module that imports ACIS headers. Its purpose:

1. **Hide ACIS-specific types** so they never appear in non- `geom` headers
2. **Encapsulate ACIS memory semantics** (`ENTITY` derivation, `lose()` lifecycle, attribute attachment) behind a clean RAIL surface
3. **Make lazy materialization (§5.4) a property of the layer**, not a discipline of every caller
4. **Preserve the kernel-swap escape hatch** at ~6–12 weeks of focused work if business conditions ever require it

8.2 Opaque handle convention

Public types in `geom` :

- `BodyHandle` — opaque 64-bit identifier; valid for the lifetime of the document
- `FaceHandle` , `EdgeHandle` , `VertexHandle` — opaque 64-bit identifiers; valid for the lifetime of their parent body
- `BodyRef` — RAII guard that holds a body materialized while the ref is alive

No public type exposes an ACIS `ENTITY*` , `BODY*` , `FACE*` , or any kernel-specific structure.

8.3 Lazy materialization contract

```
BodyRef KAL::acquire(BodyHandle h)
    // If h refers to an already-materialized body: returns immediately, increments ref count
    // If h is still SAT-only: parses SAT, materializes, increments ref count
    // BodyRef destructor decrements; if count reaches zero AND under pressure, body
    //   is unloaded back to SAT form on a worker thread

template<typename Op>
auto KAL::with_body(BodyHandle h, Op op)
    // Scoped acquire+execute+release
    // Preferred API for callers that don't need long-lived refs
```

Callers in `cmd` and `script` use `with_body` for one-shot operations. The render module holds `BodyRef` s for currently-visible bodies, releasing them as bodies move out of view.

8.4 Memory ownership rules across the KAL boundary

- The KAL **owns** every materialized kernel object
- `BodyHandle` etc. are values that callers can store freely; they do not own memory
- `BodyRef` is a refcount guard; callers must release (RAII handles this)
- No caller may `delete` a kernel object — there is no public API that returns one
- The KAL is single-threaded internally for kernel operations on a given body; cross-body parallelism is delegated to the kernel-worker pool

9. Integration with other modules

9.1 `render` and the tessellation cache

`render` subscribes to the op-stream and invalidates affected tessellation entries on each commit. The tessellation cache is keyed on `(BodyHandle, detail_level)` ; invalidation drops all detail levels for the affected body.

Render frames operate against a `Snapshot` acquired at frame start. The frame loop:

```

1. snapshot = db.snapshot()
2. visible = view_index.query(camera_frustum, snapshot)
3. for each entity in visible:
    if tessellation_cache.has(entity): submit_draw_call(entity)
    else: enqueue_tessellation(entity); submit_placeholder(entity)
4. snapshot release (automatic at scope exit)

```

The frame loop never blocks on a missing tessellation; it submits a placeholder (wireframe or low-LOD) and renders the real version on a subsequent frame.

9.2 `cmd` and transaction boundaries

Every `cmd` invocation (LISP call, .NET call, MCP tool call, UI command) maps to **exactly one** `db` transaction:

```

cmd.execute(command, args):
  1. parse + validate args
  2. txn = db.begin_transaction()
  3. try:
    run command logic; mutations go through txn
    db.commit_transaction(txn)
  except:
    db.abort_transaction(txn)
    re-raise
  4. emit op-stream record (on successful commit)
  5. surface UI feedback / return value

```

Public commitment. Every MCP tool call resolves to exactly one transaction in the host undo stack (docs/rearchitecture-plan.md §2.6). This commitment is enforced at the `cmd` boundary; no module bypasses it.

9.3 `agent` and read-only snapshots

The MCP agent operates against snapshots for read queries, and through `cmd` for mutations. The agent **never holds a snapshot for longer than 30 seconds** without re-acquiring; long-running plans re-query as they go.

The agent observes the op-stream to update its working context between queries; this is how it tracks “*what changed since I last looked*” without holding a stale snapshot indefinitely.

9.4 `net` and op-stream replication

`net` subscribes to the op-stream, batches deltas, and ships them to the document service for fan-out to co-edit peers. The local engine does not wait for ack before considering the local commit final — server authority resolves any conflicts at the geometry plane (docs/rearchitecture-plan.md §12).

`net` maintains its own checkpoint into the op-stream; local truncation of Layer 5 does not affect replication state.

9.5 script and plugin — sandboxed access patterns

Scripts (LISP, Python, JS bridge) and plugins (.NET, native) reach `db` only through `cmd`. There is no public read API on `db` exposed to plugins — they go through `cmd.query(...)` which acquires a snapshot per call.

Long-running plugin operations (more than ~1 ms estimated, or any I/O) are dispatched to a worker by `cmd`. The UI thread is never held by plugin code longer than the budget allows.

Plugin memory accounting. All plugin allocations route through the memory broker (`platform.mem_broker`) so the budget includes plugin overhead. A plugin that blows the budget is killed cleanly with a user notification — it does not bring down the engine.

10. WASM and the browser shell

10.1 The 4 GB browser memory ceiling

WebAssembly currently caps process memory at 4 GB (32-bit address space, even on 64-bit hosts). The browser shell budget defaults are correspondingly tighter:

Layer	Default native cap	Default WASM cap
Layer 2 (ACIS bodies)	50% of available	Layer 2 not active in browser — ACIS not in WASM (Phase 1–2)
Layer 3 (tessellation)	30% / 1.5 GB	25% / 800 MB
Layer 4 (indexes)	10%	10%
Layer 5 (undo log)	10% / 200 MB	10% / 100 MB

10.2 Allocator implications

We do not use `mimalloc` or `jemalloc` in WASM — Emscripten's bundled allocator is what we ship. The memory broker accounts in the same way; only the underlying `malloc` differs.

10.3 Tessellation differences (WebGPU vs native)

WebGPU has stricter buffer size limits than native APIs (typically 256 MB per buffer, vs 4 GB on Vulkan). Large meshes are chunked. The chunking is handled in `render`'s WebGPU backend; the rest of the engine is unaware.

10.4 ACIS in the browser — translator path only

No public production WASM build of ACIS exists as of writing. For the 3-year plan:

- **Phase 1–2 browser shell:** ODA's read-only translators present geometry to the browser as static meshes. No browser-side ACIS editing. The browser is a viewer + 2D editor + markup tool.
- **Phase 3 (open question):** if customer demand for browser-side 3D editing materializes, the path is either (a) bespoke ACIS WASM engineering with Spatial, or (b) server-side ACIS with the browser shipping ops to a kernel server. Decision deferred to Phase-3 planning with customer data.

11. Mobile implications (Phase 3)

11.1 iOS app memory limits

iOS imposes per-app memory limits that vary by device generation: typically 1.5–4 GB available before the OS kills the app. Memory pressure signals (`didReceiveMemoryWarning`) fire well before the kill threshold; the engine treats these as immediate eviction triggers.

11.2 Android Vulkan considerations

Android memory limits are looser but Vulkan driver overhead varies wildly by vendor. The render backend keeps GPU memory tracked separately from CPU RSS in the budget formula.

11.3 When to refuse to open

On mobile, drawings that would exceed 80% of available budget after eviction trigger a refuse-to-open with a clear explanation:

"This drawing is too large to open on this device (requires X GB; this device has Y GB available to apps). Open on desktop, or use the Lightweight View mode."

Lightweight View mode is a read-only tessellation-only mode that skips Layer 2 entirely.

12. Failure modes and recovery

Failure mode	Detection	Recovery
OOM during long ACIS operation	Memory broker returns null on allocation	Transaction aborts cleanly; pre-edit snapshot remains; user sees the loud-failure modal (\$7.5)
Tessellation cache thrash (LRU churn under tight budget)	Frame time P99 exceeds 33 ms for >5 consecutive frames	Render downshifts to a lower default detail level; user notified once per session
Worker thread crash	Worker watchdog (heartbeat-based)	Crashed worker restarted; in-flight transaction rolled back; user notified with crash report option
Plugin-induced memory pressure	Broker accounting attributes growth to a plugin	Plugin sandbox kills the plugin; engine intact; user sees plugin-killed notification
Corrupted op-stream record	Consumer-side schema validation	Replication and audit consumers checkpoint roll back; local undo continues; corruption logged for post-mortem
ACIS internal error (kernel-side exception)	KAL exception handler	Transaction aborts; KAL state cleaned via <code>lose()</code> discipline; user sees operation-failed message; bug filed with CTC
Document larger than budget on open	Pre-open size estimate	Loud-failure modal before any work is done; no partial open state to clean up
Snapshot horizon exceeded	Snapshot age tracker	Forced release; consumer re-acquires; query restarts from new snapshot

13. Telemetry and observability

13.1 What we log

Counters and histograms shipped to telemetry (per-session, aggregated):

- Open time per drawing size bucket
- Frame time P50 / P95 / P99 per session
- Peak RSS per session
- Eviction event count per layer per session
- Transaction count, commit success rate, abort rate
- Snapshot count, snapshot lifetime distribution
- Loud-failure modal count + which option the user chose
- Worker crashes (rare; high-signal)
- ACIS operation latency P99 by operation type

13.2 What we never log (PII)

- Drawing content
- Entity coordinates
- File names or paths
- Customer identifiers in raw form (hashed only)
- Plugin code or plugin-supplied data

Telemetry is opt-in by default for new installs, with a clear settings toggle. Enterprise customers may disable entirely.

13.3 The “performance health” customer-visible badge

A small status indicator in the bottom-right corner shows real-time engine health:

- **Green** — frame time on budget, memory comfortable
- **Yellow** — sustained pressure (frame P99 > 16 ms, or memory > 80% of budget)
- **Red** — pressure with active eviction; suggests user action

Clicking the badge opens a diagnostic panel with the current state of all five layers, budget usage, and eviction recent history. This is what support tickets reference instead of “*the app is slow.*”

14. Testing strategy

14.1 The DWG corpus

Fifty curated DWGs across five size / complexity buckets:

Bucket	Size on disk	Entity count	3D solid count	Count in corpus
Small	1–10 MB	1K–10K	0–10	10
Medium	10–50 MB	10K–50K	10–100	15
Large	50–150 MB	50K–200K	100–500	15
Huge	150–500 MB	200K–1M	500–2000	5
Pathological	varies	varies	varies	5 (deliberately hard)

The corpus lives in a separate licensed repository (size + licensing prevents inlining). CI pulls a curated subset for PR runs; nightly runs cover all 50.

14.2 Synthetic loops

For each corpus drawing:

- **Open-loop** — open, hold for 5s, close. Measures open time, peak RSS during open, RSS after close.
- **Interact-loop** — open, run a scripted pan-zoom-rotate-hover sequence for 60s. Measures frame time histogram, peak RSS, GPU memory usage.

- **Edit-loop** — open, run a scripted move-rotate-fillet sequence (50 operations), save, close. Measures transaction throughput, undo log growth, save time.
- **Pressure-loop** — open with budget set to 80% of the drawing's natural working set. Verifies eviction cascade triggers cleanly and the loud-failure modal appears at the right point.

14.3 Long-session soak

Nightly job: open the largest non-pathological drawing, perform 10,000 randomized edits with periodic saves, close. RSS at end must be within 5% of post-load baseline. Any growth beyond 5% is a leak and the job fails.

14.4 Sanitizers in CI

Debug-build CI runs with AddressSanitizer + LeakSanitizer enabled. Zero leaks tolerated. ThreadSanitizer enabled on a subset of tests (the MVCC machinery, the transaction queue) — too expensive for full coverage.

15. CI gates — exact specifications

15.1 Open-time gate

For each corpus drawing, open time must be $\leq$ the documented baseline $\times 1.10$ (10% regression budget). New PR fails if median across 5 runs exceeds threshold.

15.2 Frame-time gate

For each interact-loop, P99 frame time must be ≤ 16 ms steady state, ≤ 33 ms during background regen bursts. Regression beyond these = fail.

15.3 Memory ceiling gate

Per drawing class, hard RSS ceiling:

Class	Steady-state RSS ceiling
Small	500 MB
Medium	1.5 GB
Large	4 GB
Huge	8 GB
Pathological	drawing-specific

PR exceeding any ceiling = fail.

15.4 Allocation count gate

Open-edit-save loop allocation count tracked per build. A 2× increase from baseline (single PR) is suspicious — flagged for review. A 5× increase is a hard fail. This catches accidental allocator-bypass (someone forgot to use the entity pool).

15.5 Leak gate

AddressSanitizer + LeakSanitizer on every debug CI run. Any leak = fail. No suppression list except for documented third-party leaks (with reference to upstream bug).

15.6 Soak gate

Nightly. Final RSS within 5% of post-load baseline = pass. PR-level approximation: a 1000-edit shortened version of the soak that completes in ~5 minutes, with a looser 10% threshold.

15.7 Failure handling — what counts as a flake

CI gates can flake. Policy:

- **Open-time / frame-time** — flake threshold is 1 failure in 3 retries; if 2 of 3 fail, that's a real regression
- **Memory ceiling / leak / allocation count** — zero tolerance; any failure is real
- **Soak** — single-fail allowed if reproducible only at large scale; re-run on a dedicated bare-metal runner for verification

16. Tools

16.1 Tracy (always-on)

Tracy is the primary profiler. Every developer build links Tracy; running the engine attaches Tracy automatically on localhost. Frame time, allocation rate, GPU timing, and zone-based CPU profiling are visible in real time.

16.2 Intel VTune (deep dives)

For CPU-heavy investigations (kernel operation profiling, allocator hot paths). Used by the perf team during dedicated investigations, not in every build.

16.3 Heaptrack and Massif

Linux-only deep memory analysis. Used when the soak gate fails and we need to find the leak.

16.4 Visual Studio Diagnostics + WPA

Windows-specific. Visual Studio Diagnostics for CPU + memory in development; Windows Performance Analyzer for system-level traces when investigating customer-reported issues.

16.5 AddressSanitizer + LeakSanitizer

CI-enabled on every debug build. Local builds can enable via a CMake flag. ThreadSanitizer enabled on the MVCC test subset.

16.6 mimalloc / jemalloc as system `malloc`

`mimalloc` is the default native allocator (drop-in `malloc` replacement). 10–30% wins on allocation-heavy CAD workloads with near-zero integration cost. `jemalloc` is the backup if `mimalloc` surfaces issues. Neither is used in WASM (Emscripten heap only).

17. Expertise sourcing

17.1 Spatial CTC

The ACIS OEM contract bundles Corporate Technical Consulting hours (typically 20–40 hours per year). These are engineering-level consultations, not sales support. **First conversation in Phase 1: get the ACIS team to review the KAL design before we commit to it.** Worth a half-day of their time and saves quarters of trial-and-error.

17.2 External performance engineers

Toptal, Round Table Group, Guidepoint, GLG all have networks of ex-AutoCAD, ex-Revit, ex-SolidWorks, ex-Fusion performance engineers available for hourly consultations (\$300–\$800/hr). **Budget 40–80 hours in Phase 1 specifically for memory architecture review.** One day of an ex-AutoCAD perf engineer is worth a month of internal trial-and-error.

17.3 Peer back-channels

Other ACIS OEMs (Bentley, Hexagon, ANSYS SpaceClaim, BobCAD) face the same challenges. Architects at these companies are often willing to talk informally at conferences (DevCon, Spatial Summit) about KAL design and memory architecture. Not a substitute for paid consultation, but valuable for sanity-checking decisions.

18. Anti-patterns — what we will not allow

18.1 Snapshot-based undo

Storing a full document snapshot per undo entry is the simplest implementation and the wrong choice. Memory growth is catastrophic over long sessions. Delta-based undo (§5.6) is the rule.

18.2 Eager attribute loading

Loading all entity attributes when an entity is read into memory wastes work and memory. Attributes load on first access. Property panel queries one entity's attributes at a time, not all of them.

18.3 Linear-scan hit-testing

Iterating over all entities for hit-test is $O(N)$ and unusable at scale. The spatial index is mandatory. PRs that introduce a scan over entities for picking, snap, or selection are rejected.

18.4 Synchronous file I/O on the UI thread

Open, save, plot, autosave — all on workers. The UI thread does not block on disk. Ever.

18.5 Allocator-of-the-week thrash

We pick one allocator (mimalloc) and stay with it. We don't introduce per-module allocator overrides. The exception is targeted object pools for hot small-object allocation (entity handles, transient command state); those are bounded, documented, and reviewed.

18.6 Plugin direct- `db` access

Plugins go through `cmd`. There is no plugin API that returns a writable handle to a `db` entity. This is the line that, once crossed, makes every commitment above optional — so we don't cross it.

18.7 GC-based memory management in the engine hot path

RAII + opaque handles. GC introduces unpredictable pauses; pauses kill responsiveness. The Rust crates (`net` , `agent`) use Rust's ownership model, not GC; the C++ core uses RAII.

18.8 Hidden caches

Every cache declares itself to the memory broker. There are no per-function `static` caches, no thread-local growable maps, no *“oh and also we cache this little thing”*. If it's a cache, it's a registered, bounded, evictable cache.

19. Migration from IntelliCAD patterns

19.1 What we inherit (nothing)

The new engine is a clean break. We do not port IntelliCAD code. We do not port IntelliCAD patterns. We do not port IntelliCAD memory assumptions.

19.2 Patterns we deliberately don't copy

- **Single global state object accessible from anywhere** — replaced by `db` as a module with a defined public API
- **Synchronous main-loop command dispatch** — replaced by `cmd` with worker dispatch for anything long-running
- **Eager full-document load** — replaced by lazy materialization (§5.4)
- **Per-feature renderer tuning** — replaced by a single render abstraction with backend-specific implementations
- **Plugin direct database access** — replaced by `cmd`-mediated access only

19.3 What to teach engineers coming from IntelliCAD

Engineers joining from an IntelliCAD background should be onboarded with explicit teaching that:

1. There is no global “current drawing” object — you ask `cmd` for a snapshot
2. You do not modify entities directly — you submit a transaction
3. You do not iterate over all entities — you query the spatial index
4. You do not load a kernel body unless you're about to operate on it
5. You do not allocate freely — your allocation may trigger eviction, and that's the design

6. You do not block the UI thread for anything, ever

The first month for any new engineer includes a code-review pass on this document and a paired implementation of a small command end-to-end (parse args → snapshot → query → transaction → commit → op-stream emit) to internalize the pattern.

20. Phase-by-phase rollout

20.1 Phase 1 (months 0–12) — foundations

- The five-layer model is in place from commit 1
- All eight commitments enforced from commit 1
- CI gates 1–6 (open-time, frame-time, memory ceiling, allocation count, leak, soak) active by month 3
- Tracy + sanitizers wired in from commit 1
- KAL surface complete; ACIS integration complete
- The Phase-1 Windows beta to willing customers ships with the loud-failure UX in place

20.2 Phase 2 (months 12–24) — co-edit + AI integration

- The op-stream gains its replication and agent-context consumers
- Long-running snapshot patterns validated for the agent's multi-step plans
- WASM browser shell shipped with the tighter mobile-class budgets
- Lightweight View mode shipped for browser + mobile
- Performance health badge shipped to users

20.3 Phase 3 (months 24–36) — mobile + scale-out

- iOS + Android shells with mobile-class budgets
- Refuse-to-open UX validated on mobile-class devices
- Telemetry maturity — customer-reported issues triaged from the performance health badge in 80%+ of cases
- Out-of-core threshold re-evaluation: if customer drawings routinely exceed budget, scope the Phase-4 out-of-core work

21. Open questions

These are decisions deferred to specific points in the plan, not gaps in the architecture:

1. **Out-of-core / streaming database trigger.** What customer signal (P95 drawing size, frequency of loud-failure modals) tells us we need to take on out-of-core for Phase 4? Decision point: end of Phase 2, based on telemetry.
2. **Browser ACIS WASM viability.** Is full kernel editing in the browser worth the bespoke engineering with Spatial? Decision point: mid-Phase 3, based on browser-shell usage data and customer requests.

3. **Agent context window vs op-stream truncation.** Long agent sessions can exceed the agent's context window; do we summarize op-stream chunks, or rely on the agent to checkpoint? Decision point: Phase-2 agent GA planning.
4. **Snapshot horizon tuning.** Default 30s for long-running snapshots — is that too aggressive for the agent's multi-step plans? Tunable; data-driven decision in Phase 2.
5. **Memory broker accuracy on Linux cgroups v2.** Pressure signals on Linux are less reliable than other platforms — do we need a custom RSS-polling fallback? Decision point: Phase-2 Linux shell QA.

22. Glossary

- **AABB** — Axis-Aligned Bounding Box. Used for entity-level spatial indexing.
- **ACIS** — The 3D B-rep kernel from Spatial (Dassault). Our chosen kernel; see `docs/rearchitecture-plan.md` §11.2.
- **ASM** — Autodesk Shape Manager. A fork of ACIS Autodesk took private around 2001. AutoCAD's 3D solids use ASM internally; DWG persists them as ACIS-compatible SAT blobs.
- **BVH** — Bounding Volume Hierarchy. Used for view-level spatial indexing.
- **CRDT** — Conflict-free Replicated Data Type. The co-edit architecture uses CRDTs for annotations only (Layer 5 metadata plane), not for geometry. See `docs/rearchitecture-plan.md` §2.7.
- **DELA** — Distribution License Agreement. Spatial's contract terms beyond pricing.
- **DWG** — Drawing format. The canonical on-disk representation.
- **KAL** — Kernel Abstraction Layer. The `geom` module's encapsulation of ACIS.
- **LRU** — Least Recently Used. Default eviction policy for the tessellation cache.
- **MVCC** — Multi-Version Concurrency Control. The pattern that lets readers not block writers. PostgreSQL uses this.
- **op-stream** — The typed stream of committed transactions emitted by `db`. Universal substrate for undo, replication, agent, render invalidation, audit.
- **R-tree** — Spatial index data structure. Used for the entity index in `db`.
- **RAII** — Resource Acquisition Is Initialization. C++ pattern for lifetime-bound resource management. We use RAII heavily for kernel body refs, snapshots, transactions.
- **SAT** — Standard ACIS Text. The on-disk text serialization of an ACIS body. DWG stores 3D solids as SAT blobs.
- **Snapshot** — An immutable view of `db` state as of a point in time. Acquired in $O(1)$; held by every read-side consumer.
- **Working set** — The portion of process memory actively in use. What we budget against.

23. References

ODA SDKs

- [ODA Drawings SDK](#) — entity model, transaction API, spatial filter
- [ODA Visualize](#) — render abstraction, GPU buffer management

- [ODA inWEB](#) — WASM compilation path for browser shell

ACIS

- [Spatial ACIS](#) — base kernel
- [ACIS components](#) — module list (Local Operations, Healing, Defeaturing, Polyhedral, AGM, HLR, Lop, ASM)
- Spatial CTC consultation (bundled with OEM contract; first session in Phase 1 reviews the KAL design)

MVCC and database concurrency

- [PostgreSQL MVCC documentation](#) — the canonical implementation
- [SQLite WAL mode](#) — a simpler MVCC variant; useful conceptual reference

Tooling

- [Tracy profiler](#)
- [mimalloc paper](#)
- [Heaptrack](#)
- [AddressSanitizer](#)

Companion documents

- [docs/rearchitecture-plan.md](#) §16 — executive summary of this document
- [docs/rearchitecture-plan.md](#) §3 — module decomposition (which this document elaborates on for memory)
- [docs/rearchitecture-plan.md](#) §12 — collaboration architecture (Layer 5 op-stream consumer)
- [docs/rearchitecture-plan.md](#) §14 — AI agent strategy (Layer 5 op-stream consumer)
- [docs/architecture-overview.md](#) — current-state pain points that this document's commitments address